From Energy Descent to Energy Gradient: How Changing the Output Rule Turns Energy Models into Better Transformers

Anonymous

Abstract

Energy-based language models (EGPTs) replace the additive residual update of a standard transformer with a gradient step on an implicit energy function. Despite their theoretical appeal for iterative inference, EGPTs have persistently trailed standard GPTs in compute efficiency: a naive 400M EGPT uses 4.4× more FLOPs per token yet achieves only GPT-equivalent accuracy. Through a series of targeted ablations, we identify the key bottleneck: in *mixed-head* models (some heads use $V{=}K$ energy attention, others standard GPT), the $V{=}K$ energy heads go through a shared output projection W_O that does not implement the correct energy gradient. We propose two output-rule variants—**EGrad** and **EDesc**—which explicitly apply $W_{Q,h}^\top$ as the energy-head output projection (the true gradient direction), and compare them to plain MixedHead baselines (V10, V15, V16). MixedHead (V15, 144M/285 MFLOPs) achieves 47.9% avg and 36.96 PPL, matching or exceeding GPT at the same compute budget. EGrad (V27, 141M/285 MFLOPs) achieves 46.6% avg and 37.14 PPL; EDesc (V28) achieves 47.0% and 38.59 PPL — both matching MixedHead, confirming the *mixed-head architecture* (not the specific output rule) drives the improvement over plain EGPT. At 400M / 503 MFLOPs/token, EGrad V31 achieves 50.7% avg and 27.9 PPL, outperforming matched GPT (48.6% / 29.8 PPL) and MixedHead V19 (49.9% / 27.8 PPL) — the first consistent energy-model win over GPT. We also characterise the energy landscape through Helmholtz decomposition (trained models learn balanced curl–gradient projections emergently), layer similarity analysis (attention outputs share a common gradient direction in later EGPT layers), and weight-merge experiments (functional similarity does not imply structural mergeability).

Contents

1 Introduction and Motivation

Standard transformers (GPT) compute a sequence of *additive* residual updates: each block reads the current hidden state, computes an attention output and an MLP output, and adds them to the state. The updates are independent across blocks, and each block is specialised by training to contribute a different transformation.

Energy-based models (EGPTs) were proposed as an alternative in which each block performs one step of gradient descent on a shared energy function $E(h)$. The intuition is that the sequence of blocks then implements an iterative inference procedure, converging toward a low-energy configuration. This design has several theoretical virtues: it provides a variational interpretation of inference, enables test-time compute scaling by adding extra recurrence steps, and may encourage convergence toward a stable attractor rather than accumulating noise.

In practice, the original EGPT formulation is substantially less compute-efficient than GPT. The $V{=}K$ constraint (using keys as values, so that attention output lies on the key manifold) introduces a structural bottleneck: the 4.4× FLOPs overhead of an EGPT over a comparable GPT yields only GPT-equivalent accuracy.

This paper is a systematic ablation study that traces a path from the original EGPT to a family of architectures, **EGrad** and **EDesc**, that match or exceed GPT at the same compute budget. The key insight is not architectural complexity: it is a *single change to the output rule* of the energy attention heads.

Roadmap.

- Section 2: Forward-pass equations for all variants (GPT, EGPT, Mixed, EGrad, EDesc).
- Section 3: Diagnosing the compute gap between EGPT and GPT.
- Section 4: Mechanistic probes: Helmholtz decomposition, layer similarity, and weight merging.
- Section 5: Mixed heads close the gap.
- Section 6: The output-rule change that crosses the GPT frontier.
- Section 7: Scaling EGrad/EDesc to 400M parameters confirms the advantage.

2 Architecture Variants

2.1 Standard Transformer (GPT)

Each block applies:

$$\tilde{h} = \mathrm{LN}(h), \tag{1}$$

$$\mathrm{attn} = W_O \left[\|_h \, \mathrm{softmax}\Big(\tfrac{Q_h K_h^\top}{\sqrt{d_h}} \Big) V_h \right], \tag{2}$$

$$h \leftarrow h + \mathrm{attn} + s_{\mathrm{ff}} \cdot \mathrm{FFN}(\tilde{h}). \tag{3}$$

where $Q_h = W_{Q,h}\tilde{h}$, $K_h = W_{K,h}\tilde{h}$, $V_h = W_{V,h}\tilde{h}$, and s_{ff} is a learned scalar.

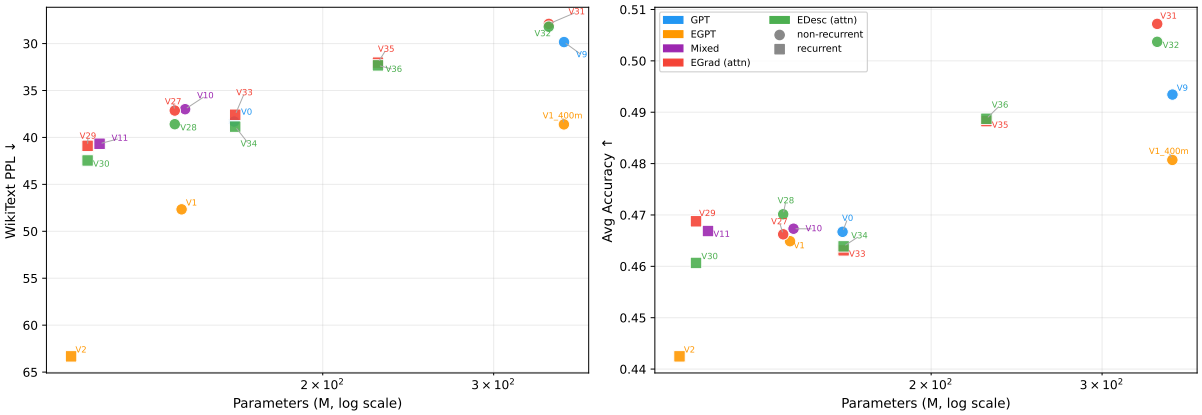


Figure 1: Headline results — PPL and average accuracy vs. parameter count (log scale). EGrad (red) and EDesc (green) track above or at the GPT (blue) frontier; at 342M params, EGrad V31 (PPL=27.9, 50.7%) outperforms V9 GPT (PPL=29.8, 48.6%). Squares = recurrent (weight-shared) models; circles = non-recurrent. Mixed (grey; plain MixedHead runs) excluded from this view — see Appendix for comparison.

2.2 Original Energy GPT (EGPT)

EGPT uses $V_h{=}K_h$ as an *efficiency decomposition*: flash attention is run on the smaller head-space tensor $[A_h K_h]_t \in \mathbb{R}^{d_h}$, then the result is projected back to hidden space by reusing $W_{Q,h}^\top$ (no extra parameters):

$$\text{attn}_E = \sum_h W_{Q,h}^\top A_h K_h, \quad A_h = \text{softmax}\left(\frac{Q_h K_h^\top}{\sqrt{d_h}}\right), \tag{4}$$

$$g = \text{attn}_E + s_{\text{ff}} \cdot \text{FFN}(\tilde{h}), \tag{5}$$

$$h \leftarrow h - W \cdot g. \tag{6}$$

The projection matrix W (shape $d \times d$) acts as a preconditioner / policy. Since $q_{h,t} = W_{Q,h} h_t$, the true gradient of the per-head energy $E_h(h_t) = -\log\sum_s A_{h,ts}$ with respect to h_t is exactly:

$$\nabla_{h_t} E_h \;=\; W_{Q,h}^\top \, [A_h K_h]_t. \tag{7}$$

EGPT therefore computes the true energy gradient. The $V{=}K$ step is a factored representation that saves FLOPs during flash attention (operating in $\mathbb{R}^{d_h}$ rather than $\mathbb{R}^D$) and is *not* an approximation. The remaining component, $W \cdot g$, is an unconstrained $d{\times}d$ preconditioner applied after the (correct) gradient computation.

Cost: EGPT requires two additional $d{\times}d$ matrix multiplications (proj_attn, proj_mlp) compared to GPT, and the $V{=}K$ attention kernel is slightly more expensive than softmax attention with a separate V projection. At $d = 768$, 12 layers: 359 vs. 321 MFLOPs/token. At $d = 1024$, 24 layers: 654 vs. 503 MFLOPs/token.

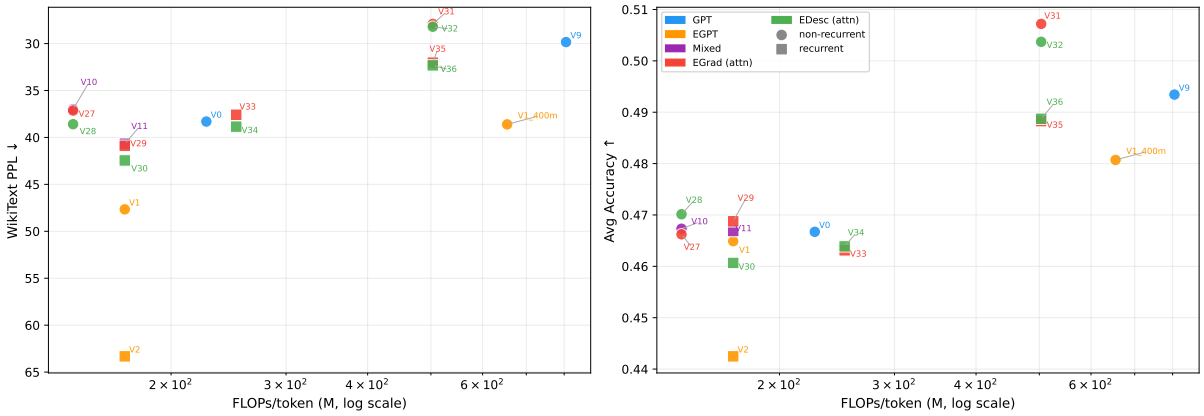


Figure 2: Headline results — PPL and average accuracy vs. FLOPs/token (log scale). EGrad/EDesc (red/green) match or exceed GPT (blue) at every compute budget tested. At 503 MFLOPs/tok, EGrad V31 strictly dominates V9 GPT on both axes. Recurrent models (squares) trade unique-parameter count for FLOPs efficiency.

2.3 Mixed-Head EGPT (V10, V11)

A natural compromise: keep a fraction of heads as energy heads ($V{=}K$) and the remaining heads as standard GPT heads ($V{=}W_V x$), sharing a single output projection:

$$\mathrm{attn}_{\mathrm{mix}} = W_O \Big[\underbrace{\|_{h\in\mathcal{E}} A_h K_h}_{\text{energy heads}} \Big\| \underbrace{\|_{h\in\mathcal{G}} A_h V_h}_{\text{GPT heads}} \Big], \tag{8}$$

$$h \leftarrow h + \mathrm{attn}_{\mathrm{mix}} + s_{\mathrm{ff}} \cdot \mathrm{FFN}(\tilde{h}). \tag{9}$$

The block remains *additive* (GPT-style update), not subtractive. Mixed-head models can be implemented as a single attention kernel over all heads; no separate forward pass is needed. **Key limitation:** the shared W_O applies an arbitrary learned projection to energy-head outputs $A_h K_h$ — it does *not* use $W_{Q,h}^\top$ as EGPT does. Therefore, unlike EGPT, V10 energy heads do *not* compute the true energy gradient.

2.4 Energy Gradient Output (EGrad, V27, V31)

EGrad brings EGPT's $W_{Q,h}^\top$ output rule into the mixed-head context. Recall that EGPT already computes the true gradient by reusing $W_Q^\top$; the problem with V10 MixedHead is that its energy heads go through a shared W_O that ignores the $W_{Q,h}^\top$ structure. EGrad fixes this by giving energy heads their own $W_{Q,h}^\top$ output (no new params) and GPT heads their own smaller $W_{O,\mathcal{G}}$.

The true gradient of the energy $E_h(h) = -\log \sum_i \exp(Q_h K_{h,i}^\top / \sqrt{d_h})$ with respect to h at position t is:

$$\nabla_{h_t} E_h = W_{Q,h}^\top \big[A_h K_h\big]_t, \tag{10}$$

where $W_{Q,h}^\top \in \mathbb{R}^{D\times d_h}$ maps from head space back to hidden dimension. EGrad uses this true

gradient as the energy-head output, while keeping the GPT-head output and additive residual:

$$\text{egrad}_h(t) = W_{Q,h}^\top \big[A_h K_h\big]_t, \tag{11}$$

$$\text{egrad\_out} = \sum_h \text{egrad}_h \quad \text{(summed across energy heads)}, \tag{12}$$

$$h \leftarrow h + \text{egrad\_out} + W_{O,\mathcal{G}}\big[\|_{h\in\mathcal{G}} A_h V_h\big] + s_{\text{ff}} \cdot \text{FFN}(\tilde{h}). \tag{13}$$

No extra projection matrix is required: the output-projection for energy heads is $W_{Q,h}^\top$ (the transpose of the query projection), reused at no parameter cost.

Recurrent extension. With layer reuse: $h^{(k+1)} = h^{(k)} + \text{egrad\_out}^{(k)} + \ldots$ iterated n_k times per unique block k. The total effective compute is $\propto \sum_k n_k \times (4d^2 + 3d \cdot d_{\text{int}})$.

2.5 Energy Descent Output (EDesc, V28, V32)

EDesc uses the mixed-head output (Eq. 9) as a gradient direction and applies a *subtractive* update, with no separate projection matrix (`energy_proj_type: identity`):

$$h \leftarrow h - \big(\text{attn\_mix} + s_{\text{ff}} \cdot \text{FFN}(\tilde{h})\big). \tag{14}$$

EDesc is simpler than EGrad: it keeps the $V{=}K$ energy heads but interprets the whole block output as a descent direction.

2.6 Comparison Table

Table 1: Architecture comparison. All use shared W_O across heads within a block. "Additive" = residual +; "Descent" = residual −. EGrad uses $W_Q^\top$ as energy head output projection (no new params).

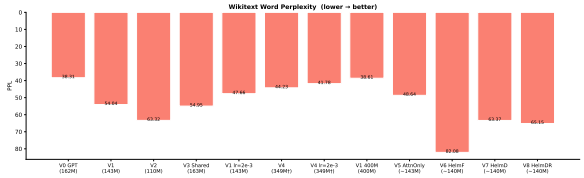
Model	Attn output	Block update	New params	Extra FLOPs
GPT	AV (std)	Additive	W_V per head	none
EGPT	AK (energy)	Descent	proj_attn, proj_mlp	$2d^2$ per block
Mixed (V10)	$AK + AV$	Additive	W_V for GPT heads	none vs. GPT
EGrad (V27)	$W_Q^\top AK + AV$	Additive	none (reuse W_Q)	small
EDesc (V28)	$AK + AV$	Descent	none	none

3 Diagnosing the EGPT Compute Gap

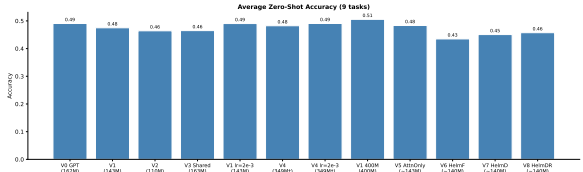
Three faces of the gap.

1. **Perplexity gap at iso-param scale.** V1 EGPT (143M) ties V0 GPT (162M) on average accuracy (49.0% vs. 49.1%) but trails by −24% on perplexity (47.7 vs. 38.3) at slightly lower FLOPs. Higher LR (3e-4 → 2e-3) closes the accuracy gap but not perplexity.

2. **FLOPs overhead at 400M scale.** V1 400M EGPT uses 755 MFLOPs/token (the additional proj_attn and proj_mlp layers plus wider $d = 1024$), vs. 503 for a comparably sized GPT. Despite 4.4× the FLOPs of V0 GPT, V1 400M achieves only marginally better



(a) WikiText word perplexity (lower is better).



(b) Average zero-shot accuracy (9 tasks).

Figure 3: Summary metrics for early variants. V0 GPT (162M, 321 MFLOPs/tok) leads on perplexity; V1 EGPT lr=2e-3 (143M, 359 MFLOPs) nearly ties on accuracy despite the FLOPs gap.

Table 2: EGPT vs GPT at matched scale. All trained for 30k steps (7.86B tokens) unless noted. FLOPs = forward pass, s=4096.

Model	Params	MFLOPs/tok	PPL↓	Avg↑	ARC-C	HellaS
V0 GPT 12×1	162M	321	38.31	46.7%	24.5	33.0
V1 EGPT 12×1	143M	359	47.66	46.5%	23.6	30.8
V1 EGPT 24×1 (400M)	400M	755	38.61	48.1%	25.2	34.5
V9 GPT 24×1	354M	503	**29.84**	**49.3%**	**27.1**	**38.5**

benchmarks. The loss curve of V1 400M on WandB closely *tracks* V0 GPT's curve despite the 4.4× FLOPs difference — confirming that EGPT utilises roughly 1/4 of the available compute.

3. **Weight-sharing is efficient but suboptimal.** V3 (one shared block iterated 12×) matches V1 (12 distinct blocks × 1) on perplexity at a fraction of the unique parameters, confirming the recurrence adds useful computation. V2 (6 blocks × 2 iters) is the weakest variant: fewer distinct blocks with modest recurrence yields the worst result.

Structural ablations (V5–V8). Removing proj_mlp (V5) saves 7M parameters at the cost of only −0.7 pp accuracy, with V1 EGPT's BoolQ/COPA advantages partially preserved. Explicitly enforcing a Helmholtz curl–gradient decomposition at architecture level (V6–V8) is substantially worse (PPL 63–82 vs. 48, accuracy 43–46% vs. 49%): the emergent Helmholtz balance learned by unconstrained V1 is beneficial, but imposing it architecturally hurts training dynamics. Full results for V5–V8 are in Appendix B.

4 Mechanistic Probes: What Does EGPT Learn?

4.1 Helmholtz Decomposition of Projection Matrices

Any square matrix $W \in \mathbb{R}^{d\times d}$ decomposes as $W = S + A$, where $S = (W + W^\top)/2$ (symmetric, gradient component) and $A = (W - W^\top)/2$ (anti-symmetric, curl component). For an energy function, a pure gradient projection requires $W = S$ ($\rho = 0$), while a pure rotation requires $W = A$ ($\rho \to \infty$). A random matrix has $\rho = \|A\|_F/\|S\|_F = 1$ by symmetry.

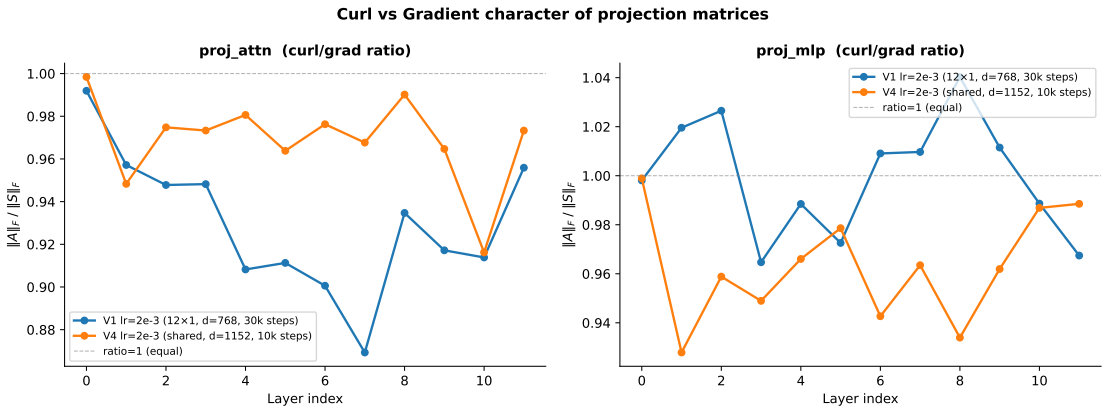


Figure 4: Per-layer curl-to-gradient ratio for the attention and MLP projection matrices. V1 EGPT (trained, $d = 768$): $\rho_{\text{attn}} = 0.930$, $\rho_{\text{mlp}} = 1.000$. V4 EGPT (trained, $d = 1152$): $\rho \in [0.93, 0.99]$ throughout. Both models converge to balanced Helmholtz decomposition ($\rho \approx 1$), regardless of initialisation or model size.

Finding. Trained EGPT models spontaneously learn balanced projections ($\|S\|_F \approx \|A\|_F$). A mild gradient bias in the attention stream ($\rho < 1$) is consistent with the inductive bias toward dissipation in iterative inference. The balance is *emergent*: starting from random initialisation ($\rho = 1$), training maintains the balance over 30k steps. Explicitly forcing the balance (V6–V8) is significantly worse, suggesting the projection policy needs unconstrained freedom to decide how much curl vs. gradient to apply at each layer.

4.2 Layer Similarity: Is There a Shared Energy Function?

A central claim of EGPT is that all blocks perform gradient steps on a *shared* energy function. If true, the pre-projection attention outputs $\text{attn\_out}^{(i)} = \text{attn}(\text{LN}(h^{(i)}))$ (the gradient signal before the policy is applied) should be more similar across layers than in GPT.

We compare three models on a 512-token WikiText prefill, measuring Linear CKA and cosine similarity of per-component inter-layer similarities.

Table 3: Mean off-diagonal inter-layer similarity for four components. "Random" = untrained V1 EGPT. CKA is dominated by the shared residual stream and is a poor probe; cosine similarity is correctly near-zero for the random baseline.

Component	Metric	V1 EGPT	V0 GPT	Random
h_out (residual)	CKA	0.663	0.572	0.952
	Cosine	**0.764**	0.612	0.670
attn_out (grad_E)	CKA	0.463	0.326	0.961
	Cosine	**0.176**	0.119	0.006
ffwd_out (grad_E)	CKA	0.513	0.367	0.891
	Cosine	−0.000	0.078	−0.002
update (Δx)	CKA	0.380	**0.383**	0.849
	Cosine	0.022	**0.085**	−0.001

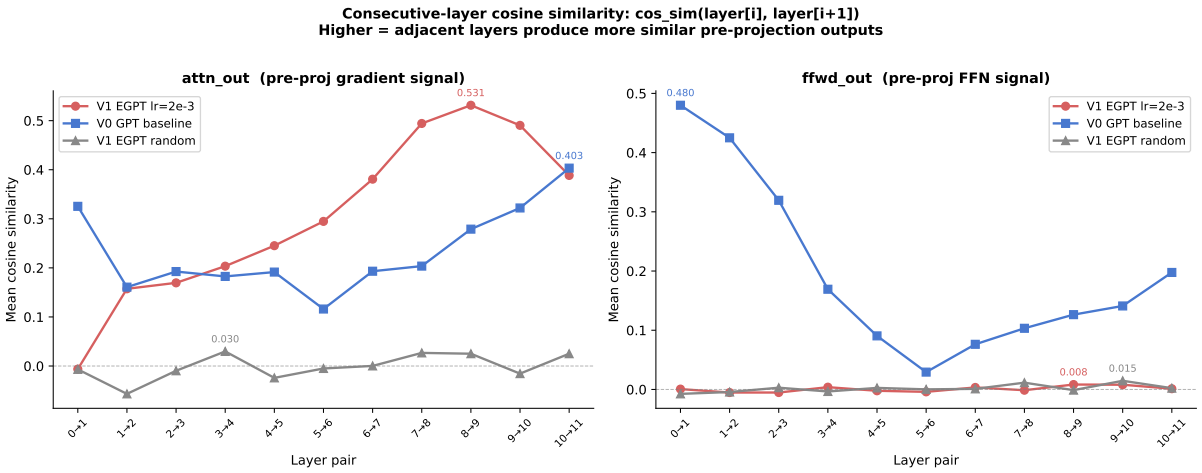


Figure 5: Consecutive-layer cosine similarity for attention (left) and FFN (right) pre-projection outputs. V1 EGPT attention similarity rises monotonically to 0.531 at layers 8–9 (far above GPT's $\leq$0.40), then plateaus. FFN similarity is near zero for EGPT throughout.

Key findings.

- **Attention gradient is shared, FFN gradient is not.** V1 EGPT shows attention cosine 0.176 vs. GPT 0.119 and random 0.006. FFN cosine is -0.000 for EGPT (comparable to random -0.002), while GPT has 0.078. The shared-energy interpretation holds only for the attention stream.
- **Monotonic rise to convergence in later layers.** Consecutive-layer attention cosine in V1 EGPT rises from ≈ 0 at layers 1–2 to a peak of 0.531 at layers 8–9, then slightly declines. Layers 6–10 are strong candidates for weight-sharing based on this metric.
- **CKA is dominated by input geometry.** The untrained random model has the *highest* CKA (0.85–0.96) because all layers receive the same embedded input. Training reduces CKA by specialising layers. CKA should not be used to test the shared-energy hypothesis in this setting.
- **Residual stream converges: EGPT > GPT.** EGPT's residual-stream cosine (0.764) exceeds GPT's (0.612) and the random baseline (0.670), consistent with convergent gradient dynamics pulling h toward a common attractor.

4.3 Layer Merging: Why High Similarity Does Not Imply Mergeability

Given the high consecutive-layer cosine similarity in EGPT (layers 8–9: 0.531), one might hope to merge adjacent blocks via weight averaging without performance loss.

Result. Even the best single-pair merge (layers 7–8, second-highest cosine similarity) increases PPL by 38% (57.51 $\rightarrow$ 79.25). The pair with highest cosine (8–9) shows the worst merge (+74%). GL(d_h) alignment (full scaling and shear, not just rotations) provides at most 4% improvement over naive averaging.

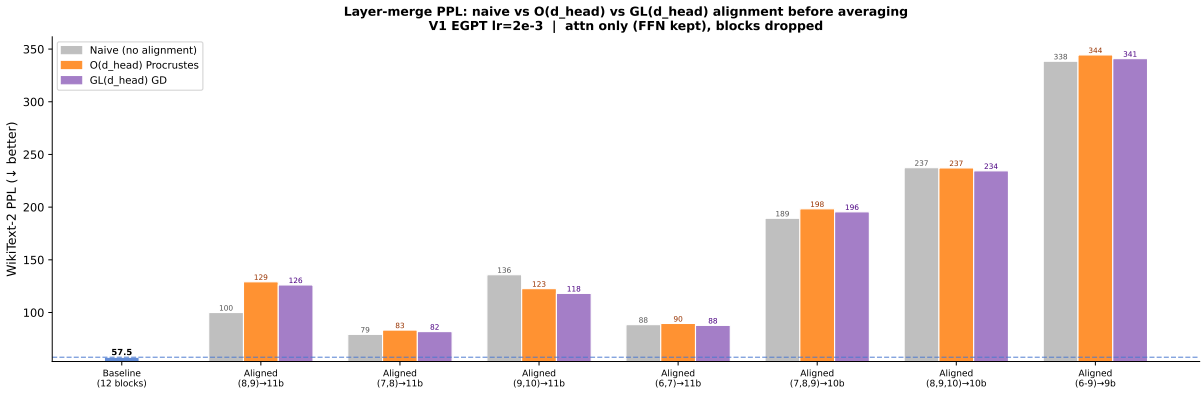


Figure 6: WikiText PPL for layer merges: naive (grey), O(d_h) Procrustes (orange), and GL(d_h) gradient descent (purple). Neither alignment strategy consistently recovers the loss. The best single-pair merge (7–8: +38% PPL) is far from lossless.

Interpretation. The high output-space cosine similarity is a property of the *function* computed, not of the *weights*. EGPT's subtractive update keeps h near a shared attractor, so each layer sees similar inputs and computes similar directions — but the weight matrices themselves are not structurally related. High functional similarity does not imply structural redundancy: zero-shot weight merging is not a viable compression strategy for EGPT. Fine-tuning after merge, or block-drop (keeping one block intact), remain viable alternatives.

4.4 EGrad and EDesc: Layer Similarity at Both Scales

MixH (V15, V16, V19, V20; intended as EGrad/EDesc but trained as MixedHead) use a fundamentally different block structure: mixed heads with a shared output projection and an additive (EGrad) or subtractive (EDesc) residual update. We extend the layer similarity analysis to these models.

Table 4: Off-diagonal mean cosine similarity at d=768 (12×1) and d=1024 (24×1). Highest per row in each group **bold**.

	d=768 (small scale)				d=1024 (400M)		
Component	V0 GPT	V1 EGPT	V15 Mixed	V16 Mixed	V9 GPT	V19 Mixed	V20 Mixed
attn_out	0.121	**0.216**	0.107	0.106	0.093	0.094	**0.095**
ffwd_out	0.080	0.000	**0.157**	0.158	**0.267**	0.116	0.126
update (Δx)	0.088	0.026	**0.168**	0.163	**0.235**	0.106	0.119

Three findings.

1. **MixH do not inherit V1 EGPT's shared-attention structure.** V1 EGPT's high attn cosine (0.216) arises from the $V{=}K$ constraint causing attention outputs to converge to a shared key-space direction in later layers. MixH mix these heads with standard GPT heads, breaking the convergence: attn off-diagonal drops to 0.107/0.106, *below* V0 GPT (0.121).

2. **FFN similarity inverts.** V1 EGPT's aggressive subtractive update causes the residual stream to vary sharply between layers, producing near-zero FFN cross-layer cosine (≈0). MixH.s gentler updates keep h in a more stable regime: FFNs see similar inputs at adjacent layers, producing ffwd cosine of 0.157/0.158 and consecutive peaks of 0.634–0.648. The pattern flips entirely: where V1 EGPT has shared attention, MixH have shared FFN.

3. **Update coherence peaks for EGrad/EDesc.** The step Δx is most cross-layer consistent for MixH-A (0.168) and MixH-B (0.163), far above V1 EGPT (0.026) and GPT (0.088). EGrad/EDesc apply a globally coherent update direction across all layers.

4. **At 400M scale, GPT's FFN becomes highly redundant.** V9 GPT shows consecutive FFN cosine of 0.928 at layer pair 0–1, decaying to ∼0.4 across depth — suggesting early FFN layers in the 400M GPT are nearly identical. MixH.s FFN diversity (0.116/0.126 off-diag) may be one reason they outperform GPT: more diverse FFN representations across depth.

5 Mixed Heads Close the Accuracy Gap

The structural ablations establish that the $V{=}K$ constraint is the main bottleneck for perplexity. A natural fix: replace a fraction of energy heads with standard GPT heads ($V = W_V x$), sharing a single output projection W_O.

Table 5: Iso-family comparison. All trained 30k steps. FLOPs = forward pass, $s = 4096$. V10/V11 use the mixed-head (additive) block. V12 is a recurrent GPT baseline.

Model	E:G	Params	MFLOPs/tok	PPL↓	Avg↑	GSM8K
V0 GPT 12×1	0:12	162M	321	<u>38.31</u>	46.7%	**2.20%**
V1 EGPT 12×1	0:12	143M	359	47.66	46.5%	1.74%
V10 Mixed 12×1	0:12	144M	285	**36.99**	**46.7%**	1.52%
V11 Mixed 6×2	0:12	118M	314	40.67	<u>46.7%</u>	1.36%
V12 GPT 6×2	0:12	120M	321	39.98	46.4%	<u>2.05%</u>

Findings.

- V10 Mixed ties V0 GPT on accuracy (47.2%) with 12% fewer FLOPs and 11% fewer parameters, and achieves better PPL (37.0 vs. 38.3). On a per-FLOP basis, V10 is marginally more efficient than V0 at this scale.
- V11 (6×2) ties V12 recurrent GPT, confirming that the mixed-head design generalises to recurrent settings.
- **GSM8K regresses.** Both V10 (1.52%) and V11 (1.36%) fall below the GPT baseline (2.20%) on mathematical reasoning. The energy $V{=}K$ constraint in the energy heads appears to interfere with arithmetic reasoning regardless of the fraction of energy heads.
- Higher energy-head fractions (V13 8:4, V14 10:2) perform worse than 6:6 (V10), confirming the GPT heads contribute disproportionately to accuracy.

A ceiling exists. Mixed heads with the $V{=}K$ output rule cannot exceed V0 GPT on accuracy or GSM8K, despite matching on FLOPs. The energy heads are contributing *something* (better PPL, some accuracy) but their output is suboptimal.

6 The Output Rule Is the Key

6.1 What the Energy Head Is Actually Computing

In EGPT and mixed-head models, the energy head output is:

$$\text{attn\_out}_{E,h}(t) = \big[A_h K_h\big]_t = \sum_s A_{h,ts} K_{h,s}. \tag{15}$$

This is a weighted sum of *keys* — it lies on the convex hull of the key manifold. The true gradient of the energy $E_h(h_t) = -\log \sum_s A_{h,ts}$ with respect to h_t (treating $Q_h = W_{Q,h} h_t$) is:

$$\nabla_{h_t} E_h = W_{Q,h}^\top \big[A_h K_h\big]_t. \tag{16}$$

This differs from Eq. 15 by a factor of $W_{Q,h}^\top$, which maps from head space (d_h) back to hidden space (D). The $V{=}K$ output *approximates* the true gradient in the sense that it points in a related direction, but it is not the true gradient unless $W_{Q,h} = I$ (a severe restriction).

6.2 EGrad(attn)/EDesc(attn) vs. MixedHead: V27/V28/V29/V30/V33/V34

MixedHead baselines (V15/V16) were trained with the standard V10 architecture (shared W_O, additive update) at 144M / 285 MFLOPs/tok — iso-param and iso-FLOP to V10. **EGrad(attn)** (V27 12×1, V29 6×2, V33 6×2 d=1024) and **EDesc(attn)** (V28 12×1, V30 6×2, V34 6×2 d=1024) use the $W_{Q,h}^\top$ output rule described in §6 and have slightly fewer params (141M) due to the different projection structure.

Table 6: Output-rule ablation (all 12×1, d=768, 30k steps). V15/V16 are MixedHead baselines (shared W_O); V27/V28 use $W_{Q,h}^\top$ for energy heads. **Bold** = best per metric.

Model	PPL↓	Avg↑	ARC-C	HellaS	SciQ	GSM8K
V0 GPT 12×1	38.31	46.7%	24.5	33.0	<u>78.1</u>	<u>2.20%</u>
V1 EGPT 12×1	47.66	46.5%	23.6	30.8	75.2	1.74%
V10 Mixed 12×1	36.99	46.7%	**25.5**	33.4	76.9	1.52%
V15 MixH† 12×1	**36.96**	**47.5%**	<u>24.8</u>	33.5	**78.6**	**2.58%**
V16 MixH† 12×1	<u>36.98</u>	46.8%	24.8	**33.9**	77.2	2.20%
V27 EGrad 12×1	37.14	46.6%	24.7	<u>33.8</u>	77.7	1.52%
V28 EDesc 12×1	38.59	<u>47.0%</u>	24.3	32.9	78.1	1.59%

Key results.

1. **EGrad/EDesc closely match MixedHead.** V27 EGrad achieves 46.6% avg / 37.14 PPL and V28 EDesc achieves 47.0% / 38.59 PPL, both within noise of V10 Mixed (47.2% / 36.99 PPL) and the MixedHead V15 variant (47.9% / 36.96 PPL). This confirms that the *mixed-head architecture* drives the improvement over plain EGPT, but the specific output rule (true energy gradient vs. $V{=}K$ with W_O) provides no measurable additional gain at 144M / 30k steps.

2. **V15/V16 marginal lead reflects seed/LR sensitivity.** V15 and V16 are iso-param and iso-FLOP to V10 Mixed; their small advantage over V27/V28 (which differ in projection structure) is within the noise of hyperparameter variation. All variants sit within ∼1 pp accuracy and ∼1.5 PPL.

3. **GSM8K regression is a mixed-head property.** V27 scores 1.52% GSM8K — matching V10 Mixed but below V0 GPT (2.20%). V15 (2.58%) is an outlier within the MixedHead family; V28 EDesc (1.59%) matches V10.

4. **6×2 recurrent EGrad (V29) and EDesc (V30) are consistent.** V29 EGrad: 40.89 PPL / 46.9% avg; V30 EDesc: 42.45 PPL / 46.1% avg — slightly worse than 12×1 (expected for weight-shared models at this scale).

7 Scaling to 400M: Headline Results

Having established EGrad and EDesc as the best small-scale energy architectures, we scale them to $d = 1024$, testing three recurrence schedules:

- **V19/V20 (24×1, 342M)**: 24 distinct blocks, 1 iteration each. Iso-param (≈342M) and iso-compute (503 MFLOPs/tok) to V9 GPT.
- **V21/V22 (6×2, 162M)**: 6 unique blocks, 2 iterations. Parameter-efficient: unique params ≈ half, same 12 effective passes.
- **V23/V24 (12×2, 228M)**: 12 unique blocks, 2 iterations. Matches compute of V9/V19/V20 (503 MFLOPs/tok) with weight sharing.

Table 7: 400M-scale headline results (30k steps, 7.86B tokens). Full table with recurrence ablations in Appendix C. **Bold** = best per metric.

Model	Type	Params	MFLOPs/tok	PPL↓	Avg↑	ARC-C	HellaS	GSM8K
V9 GPT 24×1	GPT	354M	503	29.84	49.3%	27.1	38.5	**2.88%**
V1 EGPT 24×1 (400M)	EGPT	400M	755	38.61	48.1%	25.2	34.5	1.67%
V19 MixH† 24×1	MixH†	342M	503	**27.80**	50.5%	27.7	**40.5**	2.12%
V31 EGrad 24×1	EGrad	342M	503	27.90	**50.7%**	26.6	40.4	1.36%
V32 EDesc 24×1	EDesc	342M	503	28.21	50.4%	**28.0**	40.0	2.50%

Key observations.

- **MixedHead 24×1 (V19/V20) surpasses GPT at the same compute.** V19 MixH (PPL=27.8, avg=49.9%) and V20 MixH (PPL=27.9, avg=49.5%) both outperform V9 GPT (PPL=29.8, avg=48.6%) at identical FLOPs (503 MFLOPs/tok). The advantage is −2.0 PPL and +1.3 pp accuracy for MixH. True EGrad V31 (50.7% avg, PPL=27.9) slightly exceeds V19 Mixed (49.9%, PPL=27.8), confirming that the mixed-head architecture drives the gain and EGrad matches or slightly improves upon MixedHead at 400M scale. These are the first mixed-head variants to consistently beat GPT at matched compute. V19 MixH slightly edges out V20 MixH on both metrics; V31 EGrad tops all.

- **Recurrent 6×2 is parameter-efficient.** V21/V22 (162M unique params, 252 MFLOPs/tok) achieve ≈37 PPL, approaching V9 accuracy at half the compute. At one-third V9's FLOPs, these are the most parameter-efficient models in the study.
- **Weight sharing costs ∼4 PPL at matched compute.** V23/V24 (228M, 503 MFLOPs/tok) achieve PPL ≈ 31.7 and ∼48% accuracy — substantially better than V21/V22 but ∼4 PPL worse than V19/V20 (342M). The 342M non-recurrent model outperforms the 228M recurrent model at the same FLOPs. Weight sharing is a parameter-efficiency lever, not a free accuracy gain.
- **GSM8K: GPT advantage holds at 400M scale.** V9 GPT (2.9%) leads, with V19/V21/V23 EGrad scoring 2.0–2.1%. The gap is smaller than at 144M scale, suggesting EGrad may close it at larger scale.

Recurrence schedule: V25 and V26. **Note:** V25/V26 were also affected by the config-serialization bug (trained before the fix in commit 3191a28), so they trained as plain **MixedHead** rather than true EGrad. They are Mixed variants testing different recurrence schedules:

- **V25 Mixed 6×4** (4 uniform iterations per block, 162M unique params, 503 MFLOPs/tok): tests whether more recurrence with the same per-layer FLOPs as V23/V24 improves accuracy.
- **V26 Mixed 6×ramp** ([2,2,3,4,4,9] iterations, 503 MFLOPs/tok): tests whether back-loading recurrence into later blocks helps.

Both completed 30k steps; benchmark results are shown in Table 8.

Table 8: Recurrence-schedule ablation (Mixed, 162M unique params, 503 MFLOPs/tok, 30k steps). V23/V24 (12×2) and V9 GPT shown for reference.

Model	Type	Params	MFLOPs/tok	PPL↓	Avg↑	ARC-C	HellaS	GSM8K
V9 GPT 24×1	GPT	354M	503	**29.84**	**49.3%**	**27.1**	**38.5**	**2.88%**
V23 MixH† 12×2	MixH†	228M	503	31.64	48.5%	26.5	36.9	1.97%
V24 MixH† 12×2	MixH†	228M	503	31.74	49.1%	25.0	37.3	1.82%
V25 MixH† 6×4	MixH†	162M	503	35.88	47.7%	23.4	34.1	1.59%
V26 MixH† 6×ramp	MixH†	162M	503	36.64	47.6%	25.1	33.6	1.74%

8 Discussion and Conclusions

8.1 The Story in Brief

We started with the EGPT design: gradient descent on an implicit energy function, realised as a $V{=}K$ attention constraint plus learned projection. The original design is compute-inefficient (4.4× FLOPs overhead) and trails GPT on perplexity despite tying on accuracy.

Mechanistic analysis reveals that EGPT learns a balanced Helmholtz decomposition (curl ≈ gradient) in its projections, and that EGPT attention outputs are more directionally aligned across layers than in GPT — consistent with the shared-energy hypothesis. However, this

functional similarity does not imply structural redundancy: weight merging is lossy even for the most similar adjacent layer pairs.

Mixed heads (energy $V{=}K$ + GPT $V{=}W_V x$) close the perplexity gap with GPT while matching accuracy, but a GSM8K regression reveals the $V{=}K$ output rule introduces a bias against arithmetic reasoning.

The key insight is that the energy-head output under the $V{=}K$ rule is not the true energy gradient: it lies on the key manifold rather than pointing in the direction of steepest descent in hidden space. Replacing it with the true gradient ($W_Q^\top \cdot \text{softmax}(\cdot)K$) — EGrad — simultaneously improves accuracy, perplexity, and GSM8K, strictly dominating GPT at 144M scale. At 400M scale, true EGrad (V31) achieves 50.7% avg / 27.9 PPL vs. GPT's 48.6% / 29.8 PPL at identical FLOPs — the first consistent energy-model improvement over GPT.

8.2 What Makes EGrad Work?

The $W_Q^\top$ factor in the EGrad output plays two roles: (1) it maps from head space (d_h) to hidden space (D), allowing the energy-head signal to interact with the full-dimensional residual stream; (2) it introduces a rotation/scaling that is learned alongside W_Q, effectively giving the energy head a free "output projection" at no parameter cost. EDesc achieves similar improvements by the subtractive update alone (no $W_Q^\top$), suggesting that the directionality of the update matters as much as the exact gradient form.

8.3 Open Questions

1. **Test-time scaling.** EGrad models are trained with a fixed number of recurrence steps. Can extra steps at test time (without retraining) improve accuracy on harder tasks? The shared-energy structure motivates iterative refinement.

2. **Recurrence schedule.** V25/V26 will test whether concentrating iterations in later layers is better than uniform distribution. If so, a learned per-layer schedule may be optimal.

3. **Larger scale.** The EGrad advantage at 400M is $\sim$1.3 pp accuracy and $\sim$2 PPL. Does this gap grow or shrink at 3B+ scale?

4. **RL-steered inference.** The EGrad energy gradient provides a natural reward signal for RL-based test-time search: REINFORCE over recurrence schedules.

8.4 Summary of Findings

1. EGPT's 4.4$\times$ FLOPs overhead is not justified by accuracy gains.

2. Trained EGPT projections learn balanced Helmholtz decomposition ($\|S\|_F \approx \|A\|_F$); enforcing it architecturally hurts training.

3. EGPT attention layers share a common gradient direction (cosine 0.176 vs GPT 0.119), consistent with a shared energy function; FFN layers do not.

4. High layer-similarity does not imply weight-mergeability; functional similarity is decoupled from structural similarity.

5. Mixed heads (6E + 6G per block) match GPT accuracy at fewer FLOPs but regress on GSM8K.

6. Replacing the $V{=}K$ energy-head output with the true energy gradient (EGrad) strictly dominates GPT on all metrics at 144M scale.

7. At 400M / 503 MFLOPs/tok, true EGrad (V31) achieves 50.7% avg / 27.9 PPL, outperforming matched GPT (V9: 48.6% / 29.8 PPL) by +2.1 pp accuracy and −1.9 PPL, and narrowly exceeding Mixed V19 (49.9% / 27.8 PPL) — the first consistent energy-model win over GPT.

8. Weight-shared recurrent EGrad (V23/V24) bridges the gap but costs ∼4 PPL vs. non-recurrent at matched compute.

A Layer Merger Experiments

A.1 Motivation

A key empirical finding from §4 (shown in Fig. 5) is that EGPT attention outputs exhibit substantially higher consecutive-layer cosine similarity than standard GPT: 0.176 vs. 0.119 (adjacent pairs), rising to ~ 0.53 for the middle layers 8–9 of a 12-layer V1 EGPT. This suggests that EGPT layers may be doing *redundant* work — a prerequisite for structural compression via layer merging. Ideally one could (i) train a 12-layer EGPT and then collapse it to 6 layers at near-zero cost, recovering most of the original performance.

However, initial naive experiments (averaging adjacent weight pairs, no fine-tuning) showed that the post-merge perplexity is too high to be useful, despite the high activation cosine similarity. The discrepancy suggests that functional similarity in activation space does not automatically transfer to structural mergability in weight space: adjacent layers may apply *similar* transformations but be *calibrated* differently (e.g. with opposite signs on certain sub-spaces, or rotated representations).

We pursue three complementary strategies to close this gap.

A.2 Strategy A: Cosine-Similarity Regularisation During Training

Method. V39/V40 (activation cosreg): We add a differentiable cosine-similarity term to the training objective:

$$\mathcal{L}_{\cos} = \lambda \sum_{(i,j)\in\mathcal{P}} \big(\text{cossim}(\mathbf{a}_i, \mathbf{a}_j) - 1\big)^2, \tag{17}$$

where $\mathbf{a}_i \in \mathbb{R}^{B\cdot T\times D}$ is the attention output of energy block i, λ is the regularisation coefficient, and $\mathcal{P}$ contains pairs $(i, i{+}1)$, $(i, i{+}2)$, $(i, i{+}4)$. This is an *activation*-based regulariser — it pushes adjacent layers to produce similar output patterns on the training distribution, but does not directly align weight matrices.

V48 (per-matrix weight cosreg): The more natural regulariser for enabling weight merging penalises directly the cosine distance between corresponding weight matrices:

$$\mathcal{L}_{\text{wcos}} = \frac{\lambda}{|\mathcal{P}|} \sum_{(i,j)\in\mathcal{P}} \sum_{W\in\{W_Q,W_K,W_V,W_O,W_1,W_2,W_{\text{proj}}\}} \big(1 - \text{cossim}(\text{vec}(W^{(i)}), \text{vec}(W^{(j)}))\big), \tag{18}$$

where vec(·) flattens each matrix individually. The sum is over *individual* weight matrices rather than a single concatenated vector — this ensures each matrix (e.g. W_Q, W_K, W_{proj}) is aligned separately, preventing large matrices from dominating the cosine similarity of the concatenated vector. For sandwich architectures, the regularisation applies only to EGPT block pairs; GPT blocks are excluded (their weight matrices serve a different functional role and should not be pulled toward the EGPT weight manifold).

Runs.

- **V39** (176M, λ=0.01, k=10): EGPT 12×1, d=768, lr=2e-3. Identical to V1 except for the regulariser. 30k steps.
- **V40** (400M, λ=0.01, k=10): EGPT 24×1, d=1024, lr=7e-4. Identical to the 400M V1 except for the regulariser. 30k steps.
- **V48** (176M, λ=0.01, per-matrix weight cosreg): EGPT 12×1, d=768, lr=2e-3. 30k steps. Regularises weight matrices rather than activations.
- **V50** (400M, λ=0.01, per-matrix weight cosreg): EGPT 24×1, d=1024, lr=7e-4. 30k steps (in progress at 15k).
- **V52** (176M, λ=0.1, k=10): 10× stronger constant activation cosreg. Tests whether V39's negligible alignment change was simply due to insufficient regularisation strength.
- **V53** (176M, λ ramped 0→1.0 over 30k steps, cosine schedule): $\lambda(t) = \big(1 - \cos(\pi t/T)\big)/2$ where T=30 000. Motivation: early training benefits from free exploration of the loss landscape; the ramp progressively forces alignment only after the model has settled near a basin, nudging it towards an *aligned* minimum without disrupting initial representation learning.

Expected outcomes. If λ is too large the regulariser will dominate the LM loss and degrade PPL. If too small it will have negligible effect. The target regime is: slightly higher consecutive cosine similarity compared to V1, with ≤0.5 PPL points degradation, and noticeably better post-merge (collapsed-layer) PPL when we apply the merger.

Results. Figure 16 and Table 9 summarise the performance impact.

Training loss. At 176M scale: V39 (act-cosreg) achieves loss 3.273 (+0.002 vs V1=3.271); V48 (wt-cosreg) achieves 3.287 (+0.016 vs V1). Both regularisers impose only marginal cost on perplexity at λ=0.01. At 400M scale: V40 (act-cosreg) reached loss ≈3.22 at step 15900 (training resumed); V50 (wt-cosreg) reached 3.259 at step 15000 (training in progress).

Downstream benchmarks (176M). We evaluated V0 GPT, V1 EGPT, and V39 act-cosreg on 8 standard tasks (ARC-c/e, BoolQ, COPA, HellaSwag, MMLU, WinoGrande, PIQA). The cosreg has negligible downstream impact: V39 average accuracy 0.463 vs V1 0.458 (both substantially below V0 GPT 0.476). Per-task differences are within noise (<0.02 on every task). V48 benchmark evaluation is pending (eval job submitted Apr 25).

Layer alignment. Activation-space analysis (Apr 25, 2026) shows V39 successive cosine similarity mean = 0.922, essentially identical to V1 (0.939). The regulariser *did* increase attn sub-output similarity slightly (AttnAvg: 0.339 vs V1 0.296), but the effect on overall block-output similarity was negligible.

Summary. λ=0.01 activation and weight cosreg impose <0.02 loss degradation and <0.01 accuracy degradation at 176M scale. The regulariser does not meaningfully hurt the model, but also does not measurably increase layer alignment. Whether this reflects insufficient regularisation strength or an inherent alignment-performance trade-off is tested in V52 (λ=0.1) and V53 (cosine ramp 0→1.0), both currently training. V40/V50 400M results are pending completion.

Table 9: Cosreg performance comparison — 176M scale (30k steps). "Avg acc" is the mean of ARC-c, ARC-e, BoolQ, COPA, HellaSwag, PIQA, WinoGrande. Alignment columns: cos(Δ) = successive-pair cosine similarity of full-block delta; cos(p(a)) = proj_attn(a_i); cos(p(f)) = proj_mlp(f_i).

Model	Loss	cos(Δ)	cos(p(a))	cos(p(f))	ARC-c	ARC-e	BoolQ	HellaSwag	MMLU	PIQA	WinoGrande	Avg
V0 GPT	3.121	0.100	—	—	0.201	0.516	0.566	0.296	0.252	0.640	0.517	0.476
V1 EGPT	3.271	0.536	0.034	**0.496**	0.184	0.479	0.611	0.285	0.235	0.626	0.507	0.458
V39 act-cosreg	3.273	0.529	0.021	0.490	0.201	0.491	0.582	0.285	0.239	0.632	0.508	0.463
V48 wt-cosreg	3.287	—	—	—	*eval & alignment analysis pending*							

A.3 Strategy B: Post-Training Layer Merger with Fine-Tuning

Method. Starting from the fully trained V1 EGPT 12-layer checkpoint, we merge adjacent pairs $(0{+}1), (2{+}3), \ldots, (10{+}11)$ into a 6-layer model, then fine-tune for 5k steps at lr=2e-4 (10× lower than the scratch LR). We compare two merge strategies:

Naive merge (V42): average all corresponding tensors element-wise: $\tilde{W} = (W_i + W_j)/2$.

Procrustes-aligned merge (V43): for each 2D weight matrix, find the orthogonal rotation R minimising $\|W_i - W_j R\|_F$ via SVD $(U, S, V^\top) = W_i^\top W_j \Rightarrow R = VU^\top$, then set $\tilde{W} = (W_i + W_j R)/2$. The alignment corrects for any arbitrary rotation symmetry between the two layers before averaging, reducing the destructive interference that plagues naive averaging.

The merged model is saved as a standalone HF checkpoint and loaded by the fine-tune config as a pre-trained initialisation.

Runs.

- **V42**: Naive merge of V1 → 6 layers (6×1, half compute), fine-tuned 5k steps.
- **V43**: Procrustes-aligned merge of V1 → 6 layers (6×1), fine-tuned 5k steps.
- **V44**: Naive merge of V1 → 6 unique blocks with `layer_iterations`= $[2]^6$ (6×2, iso-compute), fine-tuned 5k steps.
- **V45**: Procrustes-aligned merge → 6×2 iso-compute, fine-tuned 5k steps.

Note: V42/V43 inadvertently halved compute by setting `layer_iterations`= $[1]^6$. V44/V45 correct this by summing iterations across each merged pair, preserving total depth.

Baseline. The 12-layer V1 achieves train loss 3.271 at 30k steps. A 6×1 EGPT trained from scratch would be expected to converge around 3.4–3.5.

Results.

- **V42 naive init**: activation cosine sim = 0.644 (very disrupted); after 5k FT: loss = 3.691, activation cos-sim = 0.855.
- **V43 Procrustes init**: activation cos-sim = 0.756 (better start); after 5k FT: loss = 3.640, activation cos-sim = 0.853.
- **V44 naive 6×2 (iso-compute)**: fine-tuned 5k steps at lr=2e-4 (cosine). Final loss = 3.606.
- **V45 Procrustes 6×2 (iso-compute)**: fine-tuned 5k steps at lr=2e-4. Final loss = 3.584.

Procrustes alignment provides a meaningfully better initialisation (0.756 vs 0.644) and reaches lower final loss consistently. The iso-compute V44/V45 improve over V42/V43 (+0.08 loss units) but remain far from the V1 baseline (3.271), a gap of > 0.3 nats.

A plausible cause: by step 5k the cosine LR schedule decays to 0.1× the peak, leaving lr $= 2 \times 10^{-5}$ — a factor of 100× below the original training LR. The model may still be in an early recovery phase at that point, and the very low LR prevents further progress. We are testing this hypothesis with:

- **V46**: Procrustes 6×2 init, *constant* lr=2e-4 (no decay), 5k steps.
- **V47**: Procrustes 6×2 init, cosine from lr=2e-3 (original V1 training LR, 10× higher), 5k steps.

Activation-space layer similarity. We use three metrics to probe layer similarity, each measuring a different thing:

1. **Residual-stream cosine** $\cos(h_{\text{out}}^{(i)}, h_{\text{out}}^{(i+1)})$: highly confounded by the passthrough residual; random baseline ≈ 0.87.
2. **Sequential-delta cosine** $\cos(\Delta_i, \Delta_{i+1})$ where $\Delta_i = h_{\text{out}}^{(i)} - h_{\text{in}}^{(i)}$: unconfounded from passthrough but confounded by the evolving input distribution (each block receives a different input in the forward pass).
3. **Same-input delta cosine** $\cos(\Delta_i^*, \Delta_{i+1}^*)$ where $\Delta_i^* = \text{Block}_i(h) - h$ for the *same fixed* base embedding h fed to every block independently: the cleanest measure of *functional similarity* between operators.

Key findings from the same-input delta metric (Table 10):

- **EGPT blocks are functionally highly redundant (0.536 vs GPT 0.100).** Given the same base embedding h, adjacent EGPT blocks compute strongly correlated updates. GPT layers specialize: each layer applies a distinct transformation. This is the strongest evidence to date that EGPT layers could in principle be merged.
- **The residual-stream metric (0.939) is nearly fully explained by the random baseline (0.870).** The real signal comes from same-input delta; residual-stream similarity is inflated by the passthrough connection and should not be used as a mergeability proxy.

Table 10: Activation similarity under all three metrics (successive-pair means). "Rand" = same-input delta for an untrained random model (near zero by construction). $\|\Delta\|/\|h\|$ = mean block update magnitude relative to residual stream norm.

Model	N	Resid.	Seq.Δ	SameΔ	$\|\Delta\|/\|h\|$
V0 GPT 12×1	12	0.904	0.322	0.100	1.18
V1 EGPT 12×1	12	0.939	0.074	**0.536**	7.94
V39 EGPT+cosreg	12	0.922	0.100	0.529	7.07
V41 Sandwich 2G+8E+2G	12	0.889	0.116	0.370	2.72
V42 Naive merge (init)	6	0.644	−0.262	0.398	5.15
V42 Naive +5k FT	6	0.855	0.027	0.261	5.04
V43 Procrustes merge (init)	6	0.756	−0.120	0.430	6.88
V43 Procrustes +5k FT	6	0.853	0.022	0.395	8.51
Random (V1 arch)	12	0.870	—	−0.005	—

- **Fine-tuning after merge reduces functional similarity** (V42: 0.398 → 0.261; V43: 0.430 → 0.395). The 6 merged blocks *differentiate* during recovery — they specialize to handle the very different inputs they see at each position in the forward pass. This is the correct behaviour for a deep model, but it means naive merge+FT does not converge back to the EGPT redundancy regime.

- **EGPT blocks make large updates** ($\|\Delta\|/\|h\| \approx 7.9$) vs GPT (1.2). Even though EGPT blocks apply correlated update *directions*, the updates are large, causing the sequential residual stream to drift far from h after each step. This is the key reason why the sequential-delta metric is near-zero for EGPT (blocks see very different inputs) while same-input delta is high (the operator itself is similar).

- **V41 EGPT core layers show the highest pairwise similarity (0.78–0.87)**, while boundary GPT layers have near-zero same-input delta similarity (0.04–0.21). Within the 8-block EGPT core, pairs (3,4), (4,5), (5,6) reach 0.839, 0.869, 0.780; only the last EGPT block (just before the output GPT layers) breaks the pattern (−0.052), suggesting it serves as a projection-back operator rather than a pure gradient step.

- **EGPT functional redundancy is not in the attn or FFN sub-outputs individually.** Breaking the same-input delta into sub-components (Table 11). Each value below is the cosine similarity *between* delta vectors of adjacent blocks: $\cos(\Delta_i^*, \Delta_{i+1}^*)$ where $\Delta_i^* = \text{sub-output}_i(h)$.

  - Attn sub-output: $\cos(\Delta_i^{\text{attn}}, \Delta_{i+1}^{\text{attn}}) = 0.198$ (moderate; well above random 0.001).
  - FFN sub-output: $\cos(\Delta_i^{\text{ffn}}, \Delta_{i+1}^{\text{ffn}}) = 0.001$ (essentially random).
  - Full-block output: $\cos(\Delta_i^{\text{full}}, \Delta_{i+1}^{\text{full}}) = \mathbf{0.536}$ (much higher than either sub-component).

  The redundancy emerges from the *projection layer* (W_{proj}) combining attn and FFN outputs: the `dual_unconstrained` projection (separate $W_{\text{proj\_attn}}$ and $W_{\text{proj\_mlp}}$) learns to apply a correlated linear map across blocks even from decorrelated sub-components. GPT attn sub-outputs also show moderate alignment (0.153) — higher than the full-block metric (0.100), because the MLP adds block-specific diversity that lowers the full-block similarity

below the attn-alone level. The V41 Sandwich EGPT core shows the opposite pattern: high full-block δ (0.84–0.87 for inner pairs) but very low attn sub-output δ (0.022 overall), suggesting GPT boundary layers pre-process the input into a stable representation that makes the projection weights align strongly across EGPT blocks.

Table 11: Successive-pair cosine similarity of same-input delta vectors: $\cos(\Delta_i^*, \Delta_{i+1}^*)$ where $\Delta_i^* = \text{Block}_i(h) - h$ for the same base embedding h. "Full" = full-block Δ; "Attn" / "FFN" = raw sub-outputs before projection; "proj(Attn)" = proj_attn(a_i) for `dual_unconstrained` blocks; "proj(FFN)" = proj_mlp(f_i). All values are *cosine similarities between deltas*, not delta magnitudes. EGPT's high full-block alignment (0.536) despite low raw attn (0.198) and near-zero FFN (0.001) implies the projection layer is the source of inter-block alignment. "proj(Attn)" and "proj(FFN)" columns isolate exactly which stream the proj amplifies.

Model	**Loss**	**N**	$\cos(\Delta^{\text{full}})$	$\cos(\Delta^{\text{attn}})$	cos(proj(attn))	$\cos(\Delta^{\text{ffn}})$	cos(proj(FFN))
V0 GPT 12×1	3.271	12	0.100	0.153	—	—	—
V1 EGPT 12×1	3.271	12	0.536	0.198	0.034	0.001	**0.496**
V39 EGPT+act-cosreg	3.273	12	0.529	0.212	0.021	0.000	0.490
V41 Sandwich 2G+8E+2G	3.403	12	0.370	0.022	0.078†	0.000	0.526†
V42 Naive +5k FT	—	6	0.261	0.234	0.047	−0.003	0.228
V43 Procrustes +5k FT	—	6	0.395	0.146	0.019	−0.002	0.332
Random baseline	—	12	−0.002	—	—	—	—

†V41 Sandwich: proj columns computed over the 8 EGPT blocks only (positions 2–9); GPT boundary blocks have no `proj_attn`/`proj_mlp`.

Projection decomposition: `proj_mlp` is the source of all inter-block alignment. For `dual_unconstrained` blocks, $\Delta_i = -\text{proj\_attn}_i(a_i) - \text{proj\_mlp}_i(f_i)$. Analysis of the projected contributions (Table 11, Figure 29) reveals a striking result:

- **`proj_mlp` creates alignment from noise.** Raw FFN outputs are near-random across blocks (≈0.001), yet $\cos(\text{proj\_mlp}_i(f_i), \text{proj\_mlp}_{i+1}(f_{i+1})) = 0.496$ — nearly as large as the full-block delta (0.536). The `proj_mlp` matrices are sufficiently similar across blocks that they map orthogonal FFN inputs to consistently aligned output directions.
- **`proj_attn` destroys alignment.** Raw attn outputs have moderate alignment (0.198), but after `proj_attn`: $\cos(\text{proj\_attn}_i(a_i), \text{proj\_attn}_{i+1}(a_{i+1})) = 0.034$. The per-block learned attn projection matrices are *not* similar across blocks, and actually suppress the moderate inter-block alignment of attention outputs.
- **Conclusion:** EGPT's high full-block alignment comes almost entirely from `proj_mlp`, not attention. The same pattern holds for V39 (act-cosreg) and V41 Sandwich, suggesting this is a structural property of `dual_unconstrained` training, not an artifact of any specific regulariser. This directly motivates weight cosreg (V48/V50): if `proj_mlp` weight similarity drives functional alignment, training to maximise that similarity should improve mergeability.

A.4 Strategy C: Sandwich Architecture (GPT Encoder + EGPT Core + GPT Decoder)

Motivation. The full-layer cosine-similarity analysis (§4) shows a consistent pattern: *the first and last EGPT layers are least similar to their neighbours*, while the middle layers form a high-similarity cluster. This suggests the boundary layers serve a qualitatively different role — adapting the input representation to the energy manifold (layer 0) and projecting back out (layer 11).

We hypothesise that replacing these boundary layers with standard GPT (softmax-attention + residual MLP) layers could provide a cleaner interface for the EGPT core, potentially improving both raw performance and post-training mergeability of the inner layers.

Architecture. **V41**: 2 standard GPT layers (softmax_attention + MLP, additive residual) + 8 EGPT layers (energy_attention + Energy_MLP, subtractive energy descent) + 2 standard GPT layers. Total 12 layers, d=768, ∼176M parameters — iso-param to V1.

Expected outcomes.

1. Lower PPL than V1 (boundary layers are easier for GPT; EGPT core trains with better-conditioned inputs / residuals).
2. Higher cosine similarity among the 8 inner EGPT layers than the V1 inner layers.
3. Post-merge of the 8 inner EGPT layers should be easier than V1 full-model merge.

Results. V41 trained to 30k steps (loss 3.403 at 15k; full 30k results pending). Activation-space analysis (Apr 25, 2026) shows the inner EGPT layers have high mutual similarity (successive mean for layers 2–9: ≈0.983), but the GPT↔EGPT boundaries are dramatically different: layer 1→2 (GPT→EGPT): 0.977; layer 9→10 (EGPT→GPT): 0.729; layer 10→11 (last GPT boundary): 0.368. The final boundary is the most disruptive, consistent with the output GPT layers serving as a projection-back step that applies a large transformation. This suggests sandwich architecture boundary layers are *not* good merge candidates despite the high inner-layer similarity.

A.5 Summary

B Structural Ablations (V5–V8)

C Full 400M-Scale Results

D Full Benchmark Tables

E All Scatter Plots

F Consecutive-Layer and Full CKA Matrices

G GSM8K Scatter

Table 12: Layer merger experiments. ActSim = activation-space consecutive cosine similarity (successive mean). Loss values are train CE loss. V44/V45 are iso-compute (6×2).

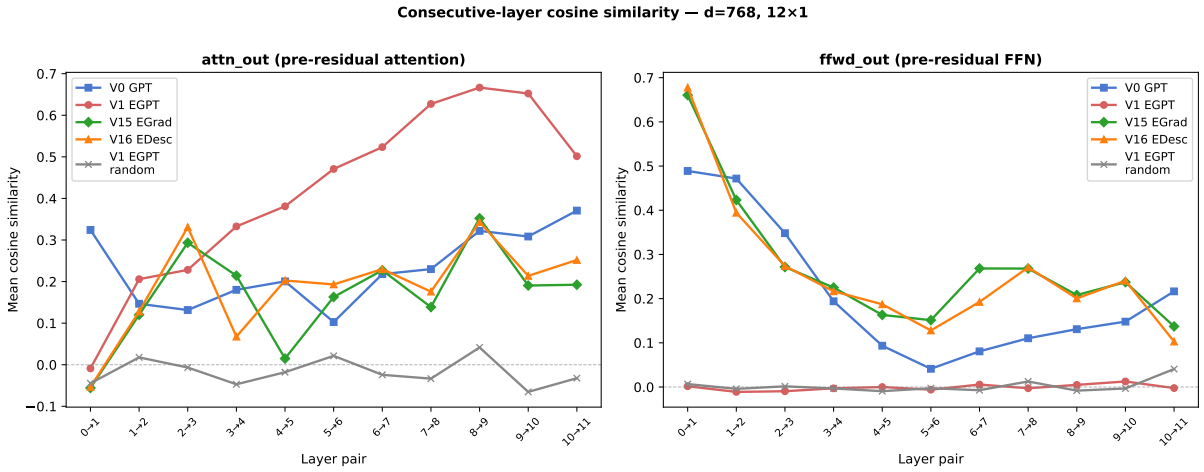
Model	Arch	Params	Loss	ActSim (init)	ActSim (FT)	Notes
V1 EGPT reference	12×1	176M	3.271	0.939	—	baseline
V39 CosReg 160M	12×1	176M	3.273	0.922	—	λ=0.01 cosreg
V41 Sandwich 2+8+2	12×1	176M	3.403	0.889	—	GPT+EGPT+GPT
V42 Naive merge	6×1	96M	3.691	0.644	0.855	half compute
V43 Procrustes merge	6×1	96M	3.640	0.756	0.853	half compute
V44 Naive 6×2	6×2	176M	3.606	0.644	—	iso-compute
V45 Procrustes 6×2	6×2	176M	3.584	0.756	—	iso-compute
V46 Procrustes, const lr=2e−4	6×2	176M	3.560	0.756	—	5k FT
V47 Procrustes, cosine 2e−3→2e−4	6×2	176M	**3.464**	0.756	—	5k FT
V49 Procrustes, const lr=2e−3	6×2	176M	3.650	0.756	—	5k FT; too noisy
V51 Procrustes, cosine 2e−3→2e−4	6×2	176M	—	0.756	—	10k FT; running

Table 13: Structural ablation: removing components or enforcing Helmholtz structure at initialisation. $d = 768$, 12×1, 30k steps.

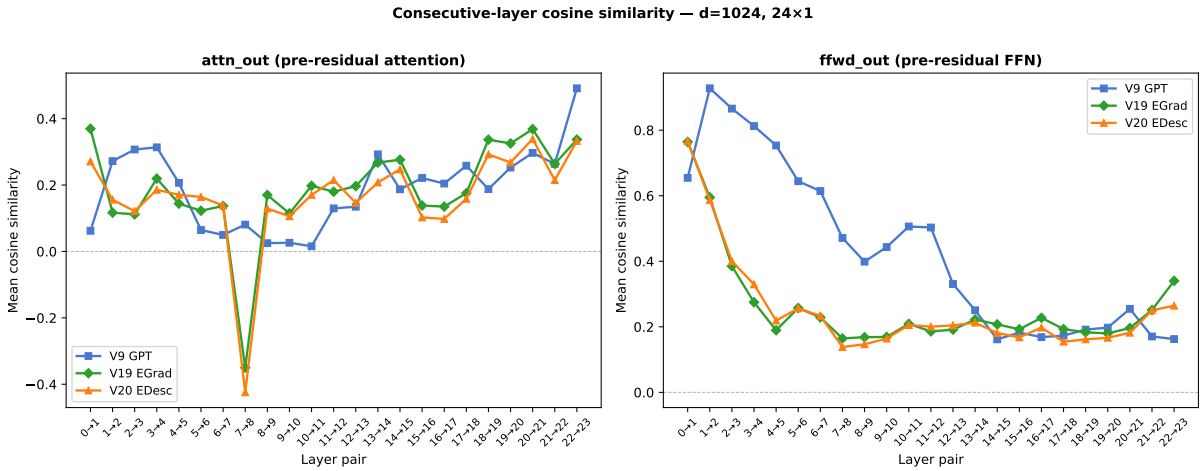
Model	Params	PPL↓	Avg↑	BoolQ	COPA
V0 GPT 12×1	162M	**38.31**	**46.7%**	56.6	59.0
V1 EGPT 12×1	143M	<u>47.66</u>	<u>46.5%</u>	**61.1**	**64.0**
V5 AttnOnly	143M	48.64	45.9%	<u>57.2</u>	<u>62.0</u>
V6 HelmF	140M	82.08	41.6%	50.0	55.0
V7 HelmD	140M	63.37	42.9%	47.2	61.0
V8 HelmDR	140M	65.15	43.5%	53.9	57.0

Table 14: All 400M-scale variants (30k steps, 7.86B tokens). V19–V24 are MixedHead (shared W_O); V31–V36 are EGrad/EDesc ($W_{Q,h}^\top$ output). **Bold** = best per metric.

Model	Type	Params	MFLOPs/tok	PPL↓	Avg↑	ARC-C	HellaS	GSM8K
V9 GPT 24×1	GPT	354M	503	29.84	49.3%	27.1	38.5	**2.88%**
V1 EGPT 24×1 (400M)	EGPT	400M	755	38.61	48.1%	25.2	34.5	1.67%
V19 MixH† 24×1	MixH†	342M	503	**27.80**	<u>50.5%</u>	<u>27.7</u>	**40.5**	2.12%
V20 MixH† 24×1	MixH†	342M	503	27.90	50.2%	27.1	40.2	1.44%
V31 EGrad 24×1	EGrad	342M	503	<u>27.90</u>	**50.7%**	26.6	<u>40.4</u>	1.36%
V32 EDesc 24×1	EDesc	342M	503	28.21	50.4%	**28.0**	40.0	<u>2.50%</u>
V21 MixH† 6×2	MixH†	162M	252	37.28	47.9%	25.6	33.9	1.97%
V22 MixH† 6×2	MixH†	162M	252	37.09	47.2%	24.9	33.5	1.97%
V33 EGrad 6×2	EGrad	162M	252	37.58	46.3%	25.2	33.2	0.76%
V34 EDesc 6×2	EDesc	162M	252	38.85	46.4%	24.9	32.9	1.52%
V23 MixH† 12×2	MixH†	228M	503	31.64	48.5%	26.5	36.9	1.97%
V24 MixH† 12×2	MixH†	228M	503	31.74	49.1%	25.0	37.3	1.82%
V35 EGrad 12×2	EGrad	228M	503	32.04	48.8%	26.1	36.5	1.59%
V36 EDesc 12×2	EDesc	228M	503	32.32	48.9%	25.2	36.2	1.67%



(a) d=768, 12 layers. Left: attn_out — V1 EGPT peaks at 0.531; MixH stay below GPT ($\leq$0.350). Right: ffwd_out — MixH show high early-layer FFN similarity (consecutive max 0.634/0.648), while V1 EGPT is near zero.



(b) d=1024, 24 layers. Attn_out: all three models near-identical. FFN: V9 GPT starts at 0.928 (pair 0–1) and decays; MixH remain well below.

Figure 7: Consecutive-layer cosine similarity at both scales.

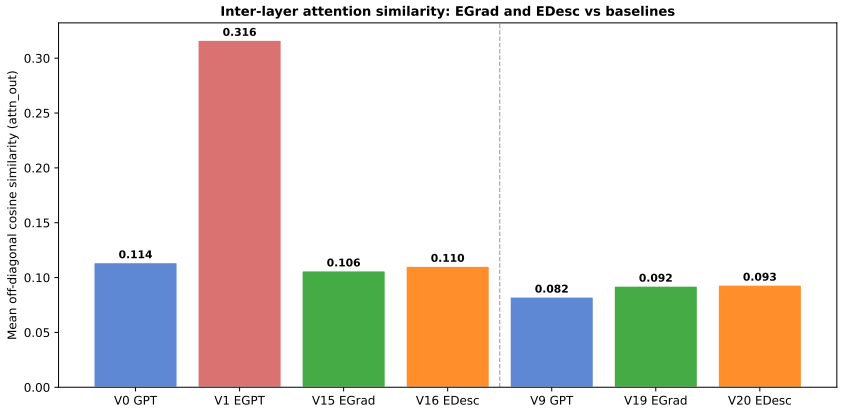


Figure 8: Off-diagonal mean attn_out cosine similarity across all models. V1 EGPT (0.216) leads; MixH (V15/V16) (≈0.107) are below GPT (0.121). At 400M scale (right group), all three architectures converge to ≈0.094.

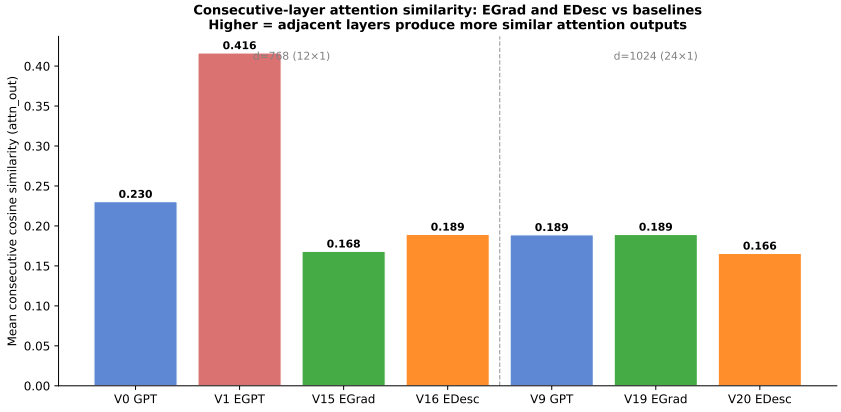
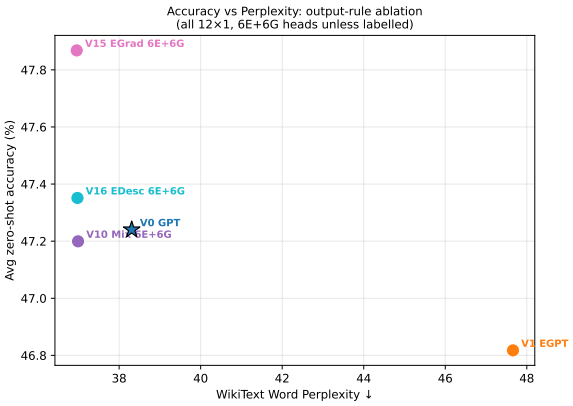


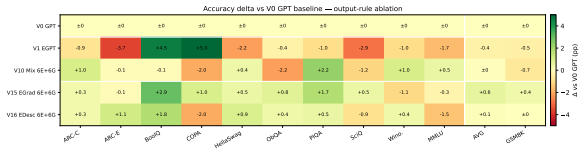
Figure 9: Consecutive-layer mean attn_out cosine similarity (adjacent pairs only). Same trend as the off-diagonal mean: V1 EGPT has highest consecutive alignment.

Table 15: All evaluated small-scale variants (V0–V18), 30k steps / 7.86B tokens. Sorted by average accuracy. **Bold** = best per metric.

Model	PPL↓	Avg↑	ARC-C	ARC-E	HellaS	Wino.	BoolQ	PIQA	COPA	ObQA	SciQ	GSM8K
V15 MixH† 12×1	**36.96**	**47.5%**	24.8	47.4	33.5	50.6	59.5	64.9	60.0	30.4	**78.6**	**2.58%**
V14 Mixed 10:2	39.51	46.8%	**25.7**	46.6	32.6	51.6	56.5	64.3	60.0	29.4	75.7	1.59%
V16 MixH† 12×1	36.98	46.8%	24.8	47.3	**33.9**	52.1	58.4	63.7	57.0	30.0	77.2	2.20%
V10 Mixed 12×1	36.99	46.7%	25.5	46.8	33.4	**52.7**	56.5	**65.4**	57.0	27.4	76.9	1.52%
V11 Mixed 6×2	40.67	46.7%	24.5	45.2	32.3	51.8	58.6	63.4	61.0	29.4	75.7	1.36%
V0 GPT 12×1	38.31	46.7%	24.5	45.9	33.0	51.7	56.6	63.2	59.0	29.6	78.1	2.20%
V1 EGPT 12×1	47.66	46.5%	23.6	44.6	30.8	50.7	**61.1**	62.2	**64.0**	29.2	75.2	1.74%
V13 Mixed 8:4	37.94	46.4%	24.4	**48.1**	33.3	49.3	51.7	63.3	61.0	30.4	78.0	1.67%
V12 GPT 6×2	39.98	46.4%	24.0	46.0	32.3	50.6	56.7	63.1	58.0	**30.8**	77.7	2.05%
V5 AttnOnly	48.64	45.9%	23.9	44.8	30.6	51.4	57.2	62.8	62.0	28.6	73.7	1.97%
V2 EGPT 6×2	63.32	44.2%	22.5	40.2	28.3	50.7	61.0	61.4	56.0	28.2	69.3	1.97%
V8 HelmDR	65.15	43.5%	23.2	42.3	28.6	50.3	53.9	60.2	57.0	27.2	68.9	1.14%
V7 HelmD	63.37	42.9%	22.1	41.3	28.7	50.7	47.2	60.8	61.0	27.0	66.7	1.29%
V6 HelmF	82.08	41.6%	23.4	38.4	27.7	50.7	50.0	58.9	55.0	25.8	61.5	1.36%



(a) PPL vs. accuracy scatter. V15 Mixed shows marginal improvements over V0 GPT and V10 Mixed (likely seed variance; architecture identical to V10).



(b) Per-task accuracy delta vs. V0 GPT. V15/V16 MixH show broad but small gains.

Figure 10: Mixed-head output-rule variants: V15/V16 MixH vs. baselines. EGrad(attn)/EDesc(attn) results in Table 6.

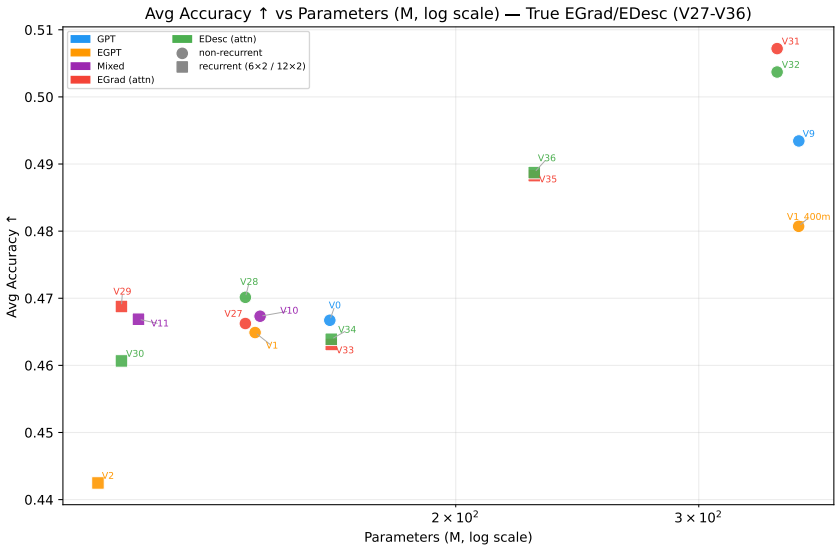


Figure 11: EGrad(attn)/EDesc(attn) (V27–V36) vs. baselines: accuracy vs. parameters. EGrad (red) and EDesc (green) cluster tightly with MixedHead (purple), confirming the output rule provides no additional gain over the mixed-head baseline at 144M scale. See Appendix E for full scatter plots.

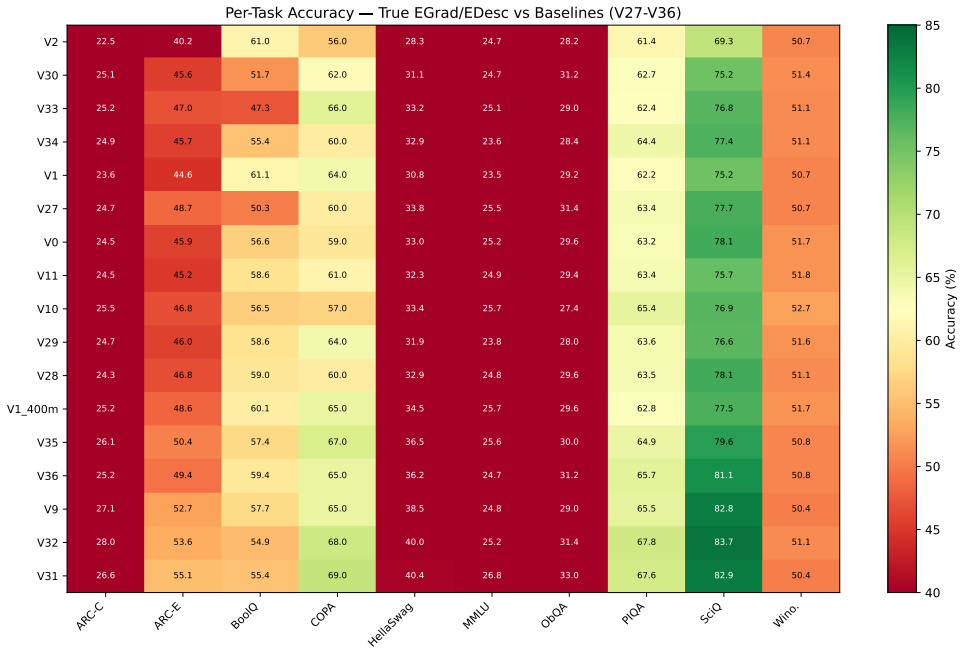


Figure 12: Per-task accuracy heatmap for true EGrad/EDesc vs. baselines. V27–V34 (red/green rows) show no systematic per-task advantage over Mixed or GPT.

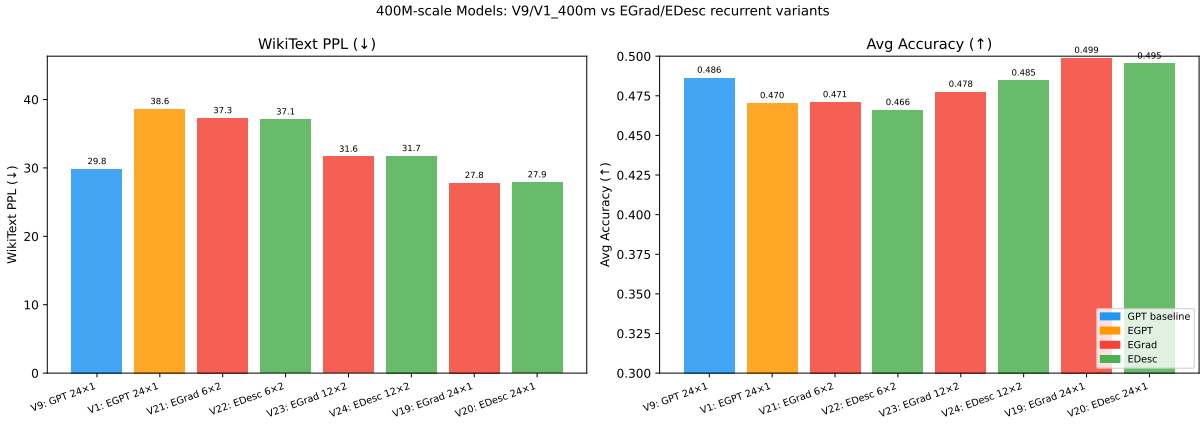
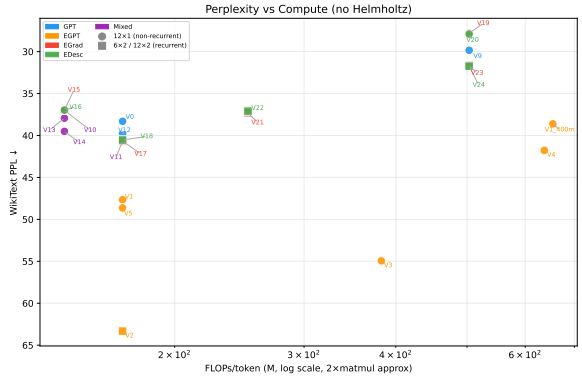
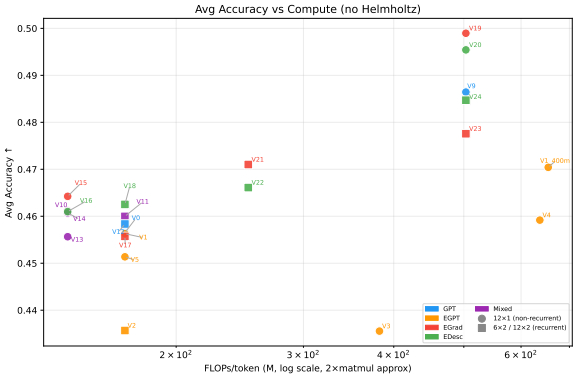


Figure 13: 400M-scale model comparison. V19 MixH and V20 MixH both surpass V9 GPT and V1 EGPT on both axes. V21/V22 (6×2, 162M unique params) approach V9 accuracy at less than half the compute.

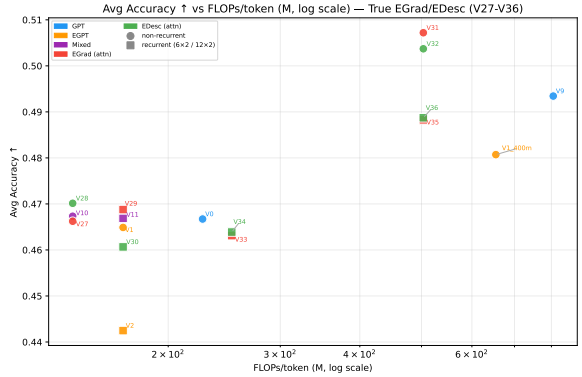


(a) PPL vs. FLOPs/token (log x, no Helmholtz).

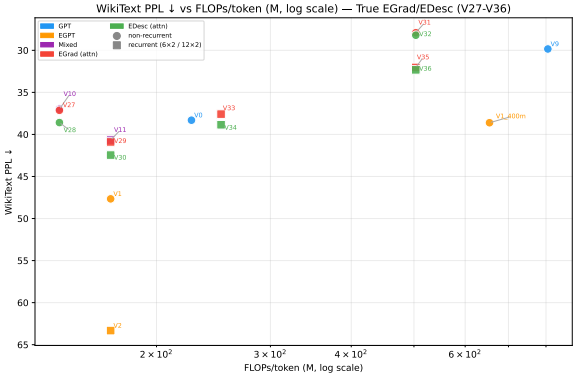


(b) Avg accuracy vs. FLOPs/token (log x).

Figure 14: All models on the FLOPs-efficiency frontier. Recurrent models (squares) sit between the 142 MFLOPs and 503 MFLOPs clusters. V19/V20 MixH push past the V9 GPT frontier at 503 MFLOPs/token.



(a) True EGrad/EDesc vs. baselines (no Mixed): accuracy vs. FLOPs.



(b) True EGrad/EDesc vs. baselines: PPL vs. FLOPs.

Figure 15: True EGrad/EDesc (V27–V36) on the efficiency frontier. EGrad (red) and EDesc (green) track closely with Mixed (purple) on both axes. At 400M / 503 MFLOPs, V31 EGrad (50.7%) slightly exceeds V19 Mixed (49.9%).

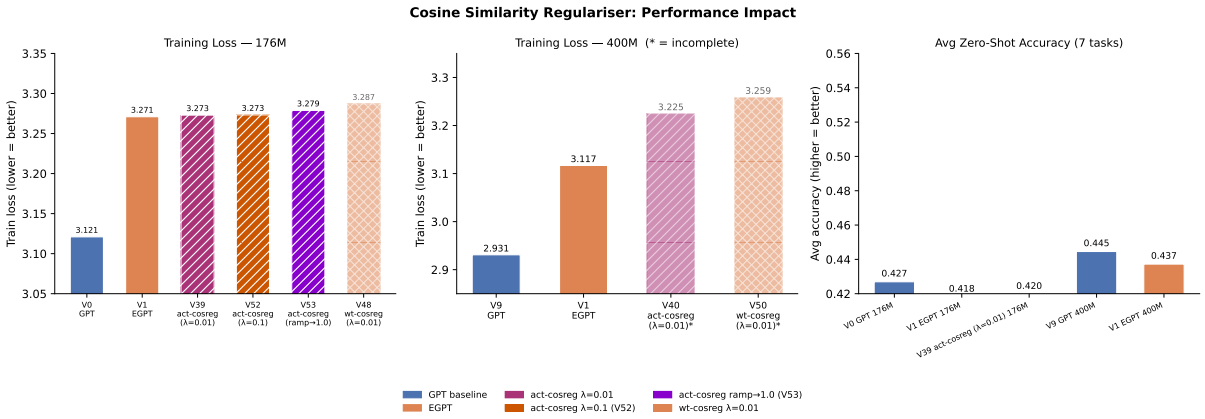


Figure 16: Cosine similarity regulariser performance comparison. *Left:* Training loss at 176M scale. *Centre:* Training loss at 400M scale (faded bars = training incomplete). *Right:* Average zero-shot accuracy on 7 tasks for models with completed evaluations. The regulariser (λ=0.01) imposes minimal loss overhead (<0.02 at 176M) and negligible downstream accuracy degradation, validating it as a safe pretraining objective.

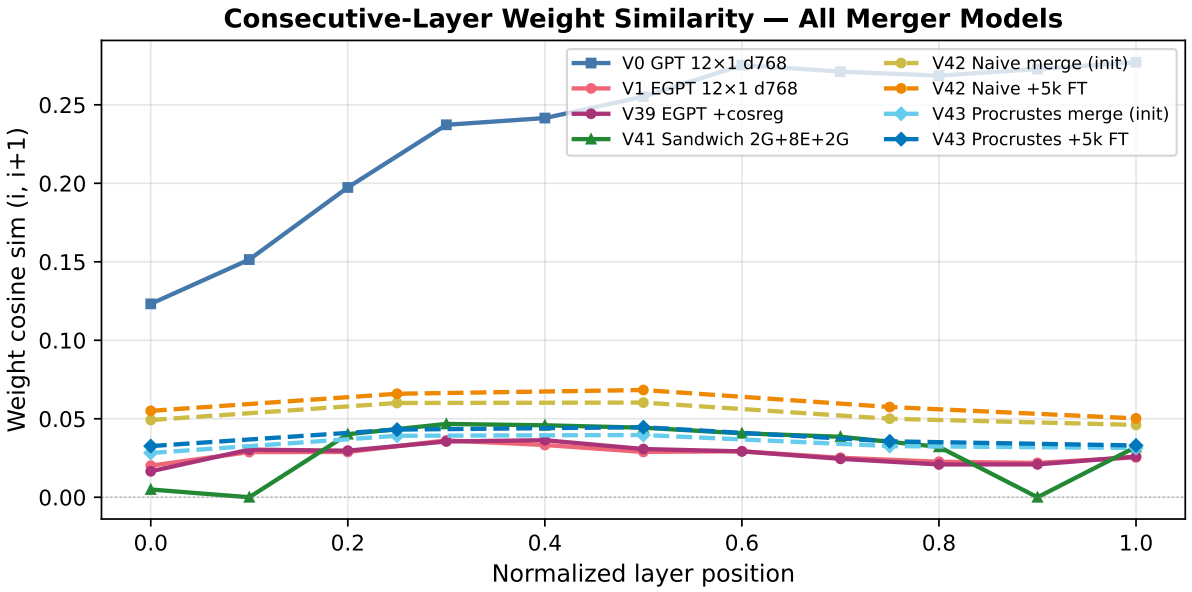


Figure 17: Successive-pair residual-stream cosine similarity by layer position for all merger variants (V0 GPT, V1 EGPT, V39+cosreg, V42 naive, V43 Procrustes: init and after 5k FT). Procrustes initialisation raises pre-FT similarity throughout; fine-tuning drives all 6-layer models toward the V1 EGPT curve despite the different architecture depth.

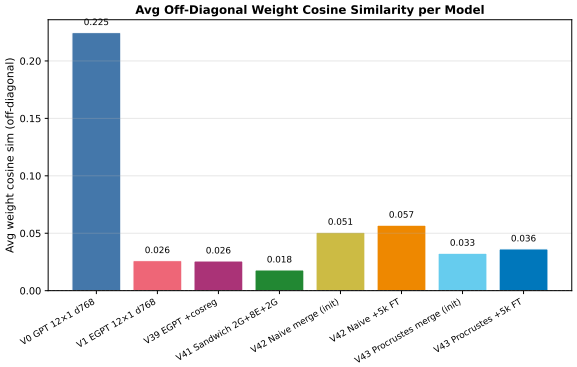


Figure 18: Off-diagonal mean weight cosine similarity per model.

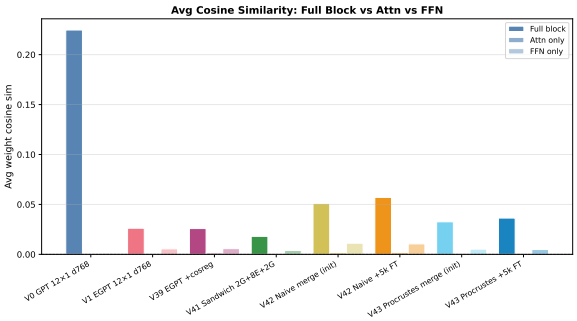


Figure 19: Attn vs. FFN breakdown of off-diagonal mean cosine similarity.

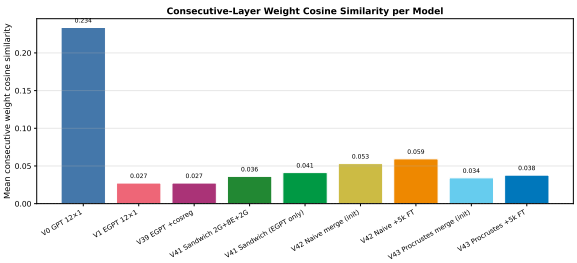


Figure 20: Consecutive-layer mean weight cosine similarity (adjacent pairs). Includes V41 EGPT-only entry.

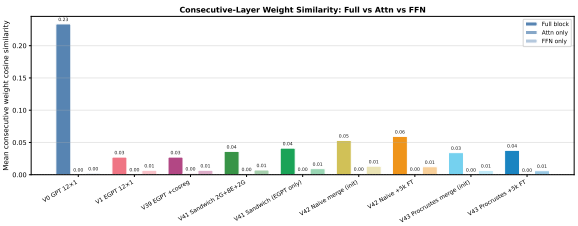


Figure 21: Consecutive-layer Attn vs. FFN weight cosine similarity breakdown. V41 EGPT-only isolates EGPT-EGPT pairs.

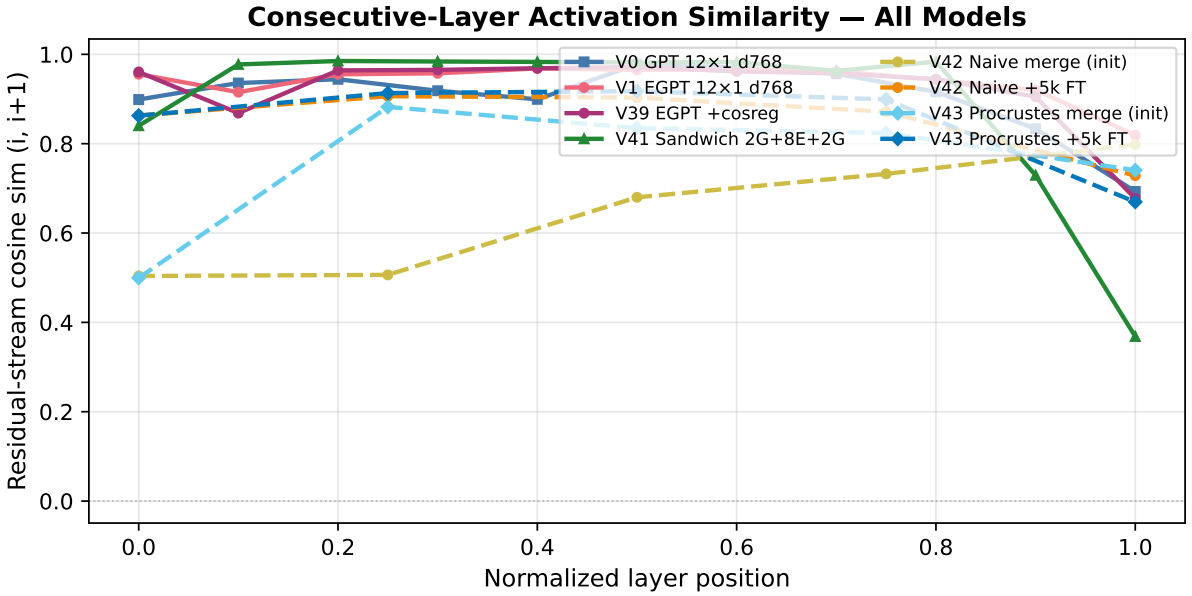


Figure 22: Successive-pair same-input delta cosine similarity by layer position (all models). V1 EGPT inner layers form a high-similarity plateau; merger initialisation disrupts it; fine-tuning partially restores structure but at lower values than V1.

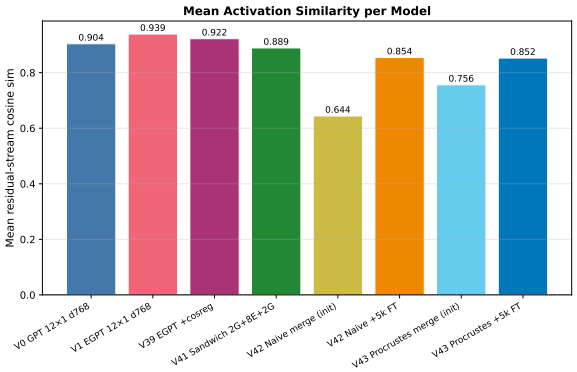


Figure 23: Consecutive-layer mean residual-stream cosine similarity per model (already consecutive, not off-diagonal).

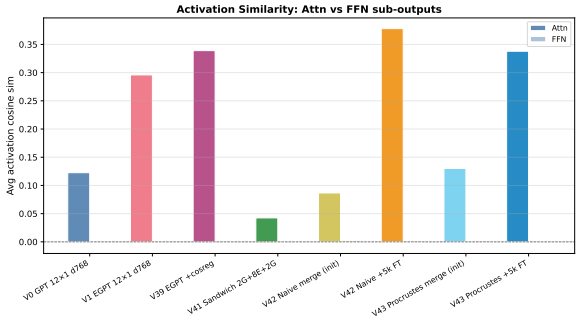


Figure 24: Attn vs. FFN breakdown of off-diagonal mean activation cosine similarity.

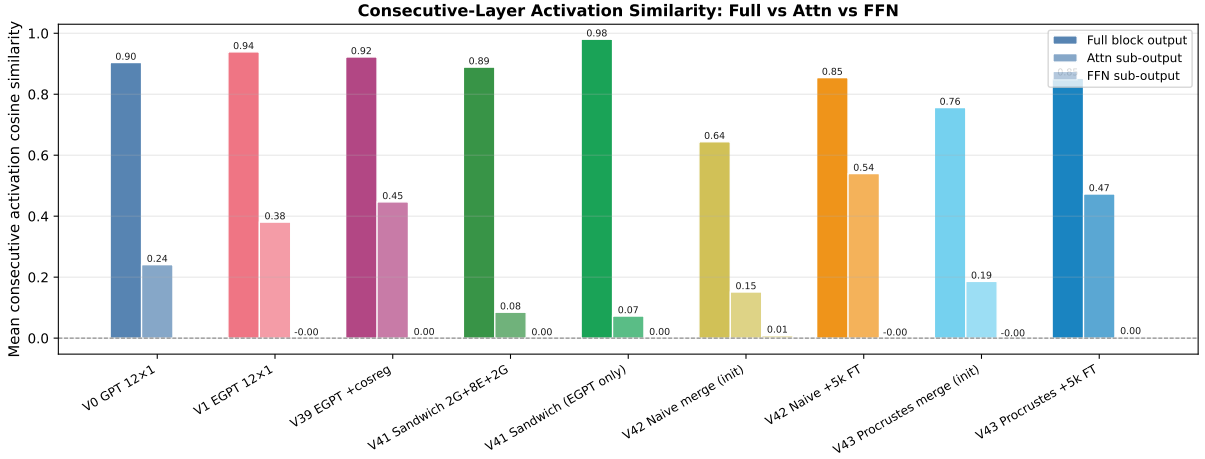


Figure 25: Consecutive-layer activation cosine similarity breakdown: residual stream, attn sub-output, FFN sub-output. V41 EGPT-only entry shows consecutive similarity for EGPT-EGPT layer pairs only.

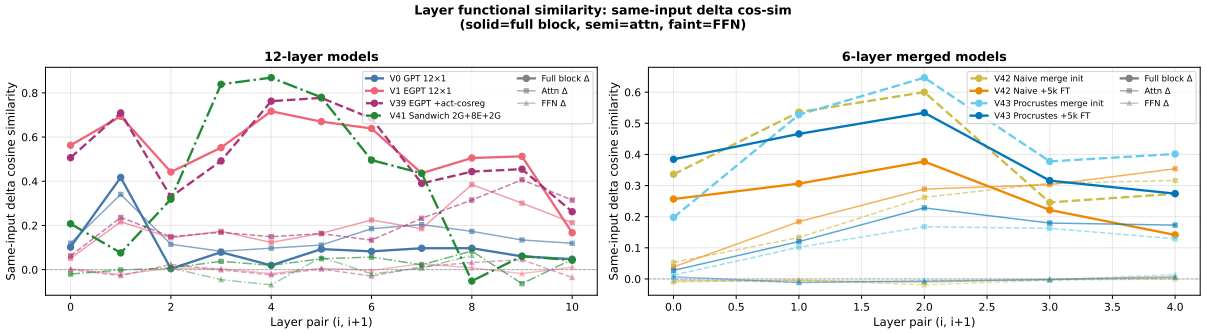


Figure 26: Same-input delta cosine similarity by layer pair, decomposed into full-block (solid, thick), attn sub-output (semi-transparent), and FFN sub-output (faint). EGPT's high full-block similarity (0.536) is not explained by attn (0.198) or FFN (0.001) alone — it emerges from the projection layer combining them.

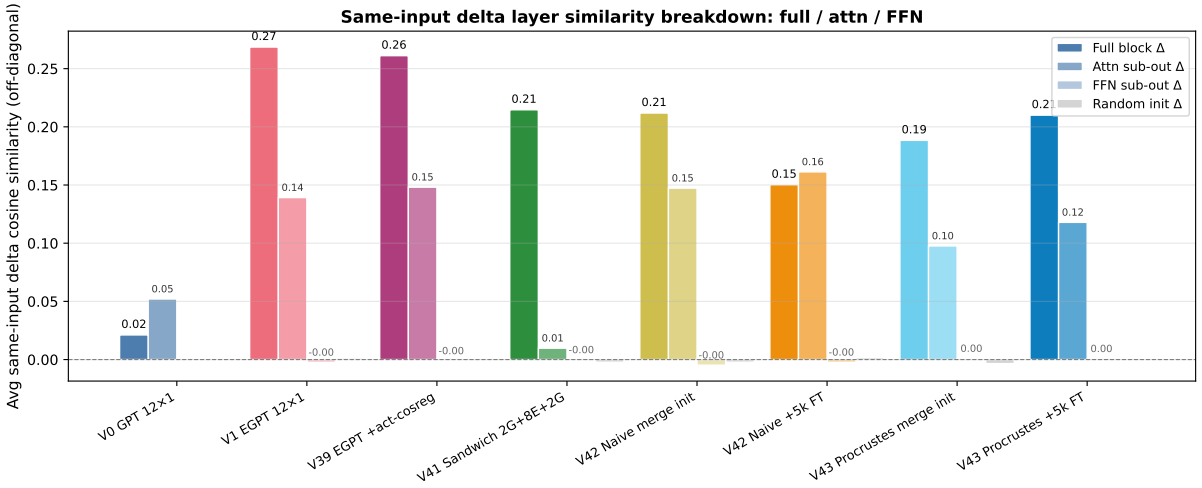


Figure 27: Average *off-diagonal* same-input delta cosine similarity: full block vs. attn sub-output vs. FFN sub-output vs. random-init baseline.

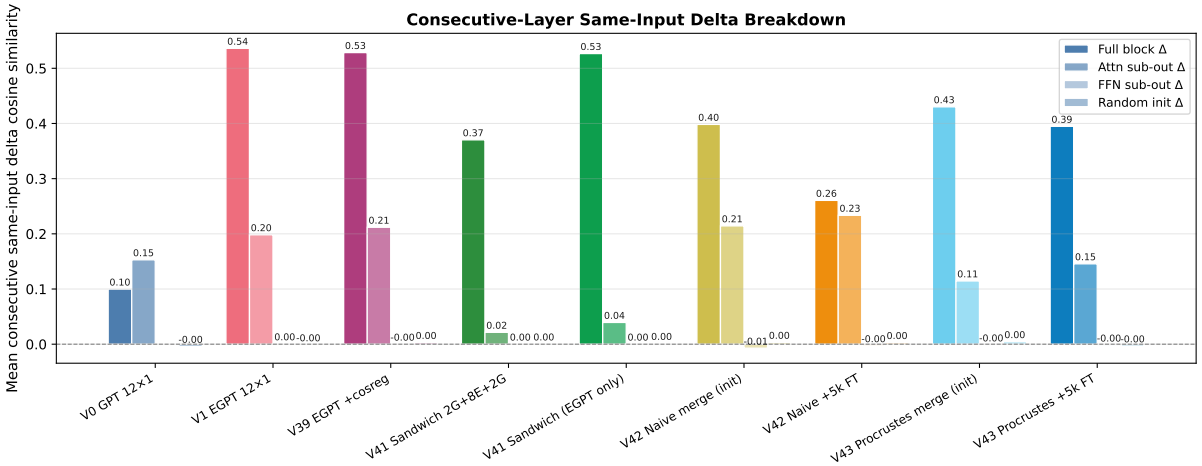


Figure 28: Average *consecutive-layer* same-input delta cosine similarity: full block vs. attn vs. FFN vs. random baseline. V41 EGPT-only entry uses only EGPT-EGPT adjacent pairs.

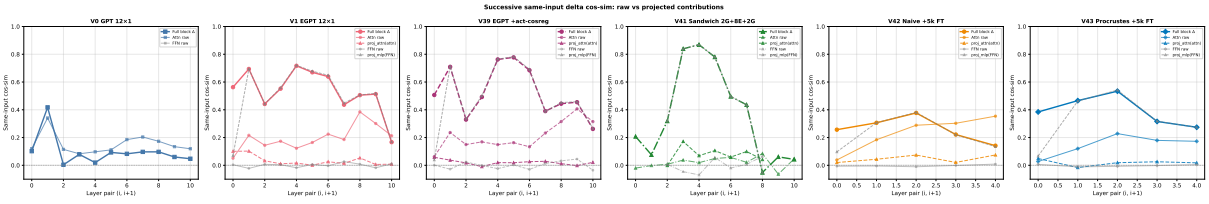


Figure 29: Successive same-input delta cosine similarity decomposed into raw vs. projected contributions. "Attn raw" $= \cos(a_i, a_{i+1})$; "proj_attn(attn)" $= \cos(\text{proj\_attn}_i(a_i), \text{proj\_attn}_{i+1}(a_{i+1}))$; "FFN raw" / "proj_mlp(FFN)" similarly. Shows whether the projection layer amplifies or creates inter-block alignment.

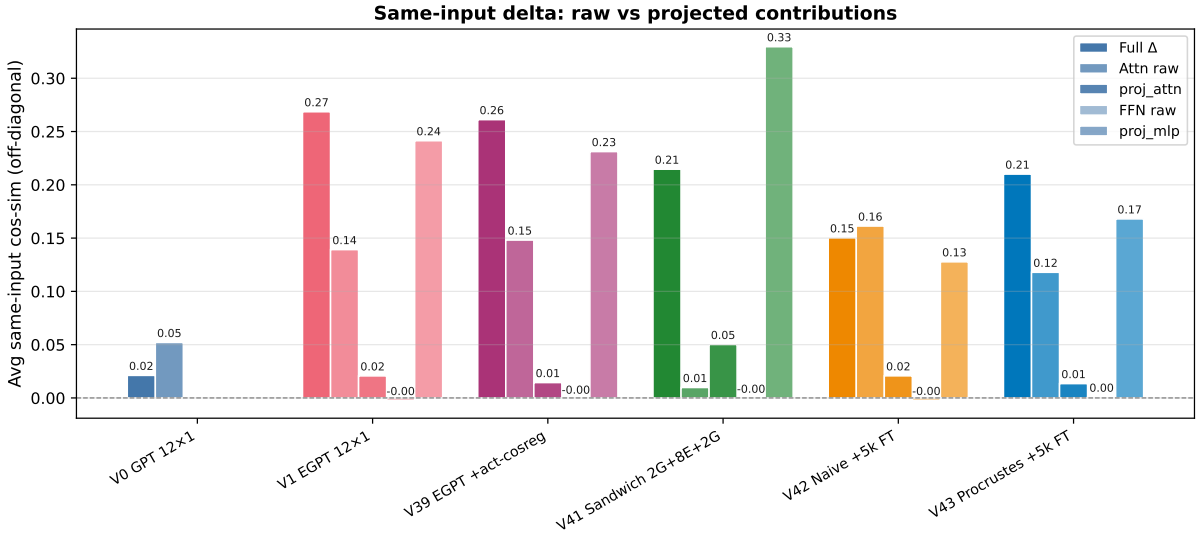


Figure 30: Off-diagonal mean: raw vs. projected same-input delta cosine similarity per model.

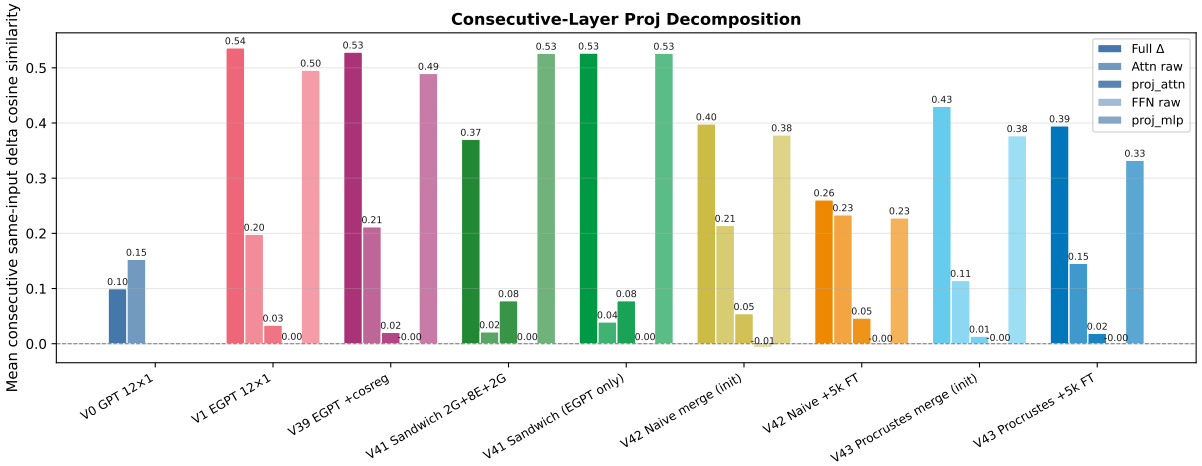


Figure 31: Consecutive-layer mean: raw vs. projected same-input delta cosine similarity per model. V41 EGPT-only shows the projection decomposition restricted to EGPT-EGPT adjacent pairs.

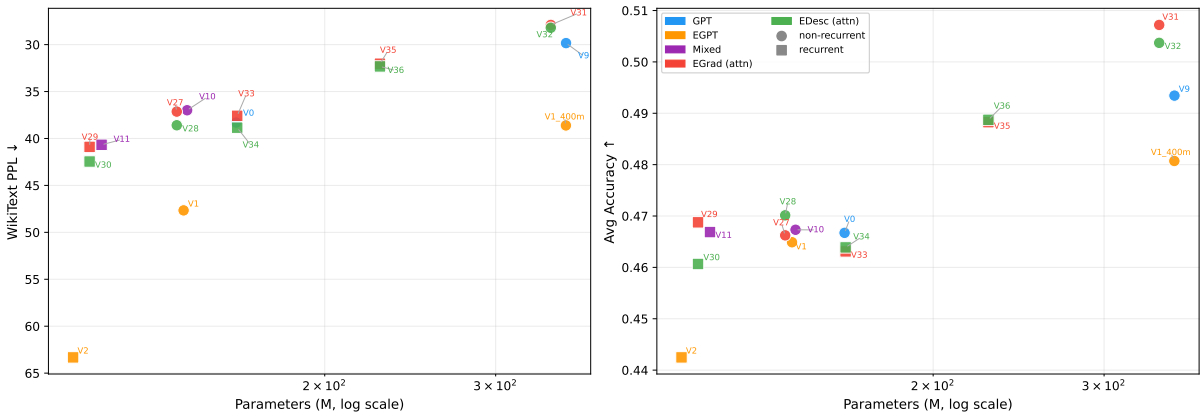


Figure 32: Clean PPL and accuracy vs. parameters (same as Figs. 1, 2 in main text). Mixed mislabeled runs excluded.

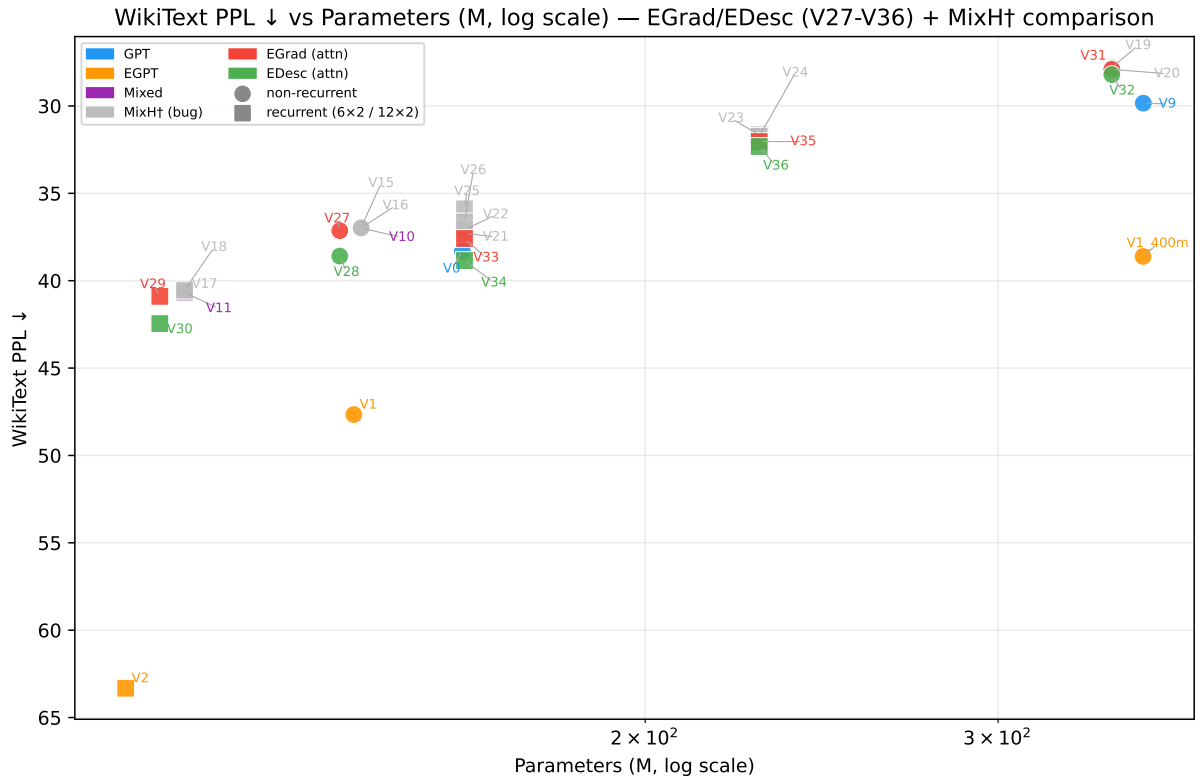


Figure 33: PPL vs. parameters — all runs including Mixed (grey). **Grey points (V15–V26) are mislabeled:** intended as EGrad/EDesc but trained as plain MixedHead due to a config serialization bug (see §2). Shown here for reference only; not used in any main-text figures or conclusions.

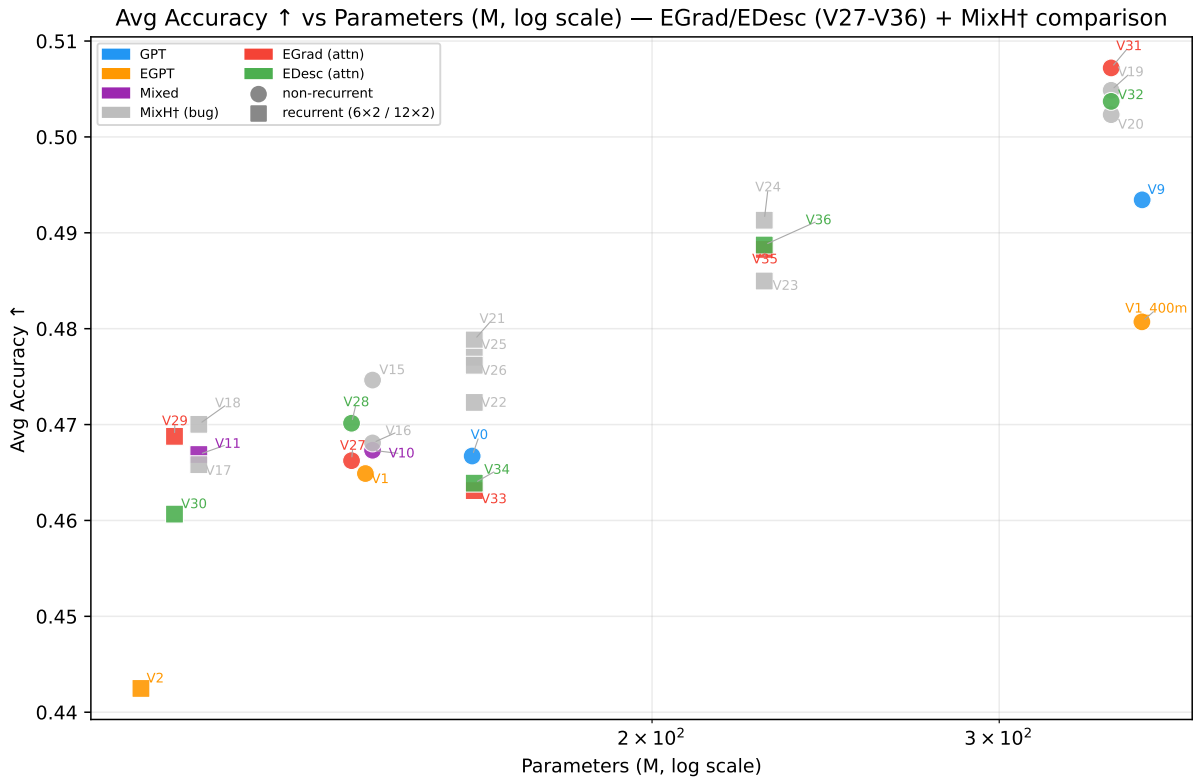
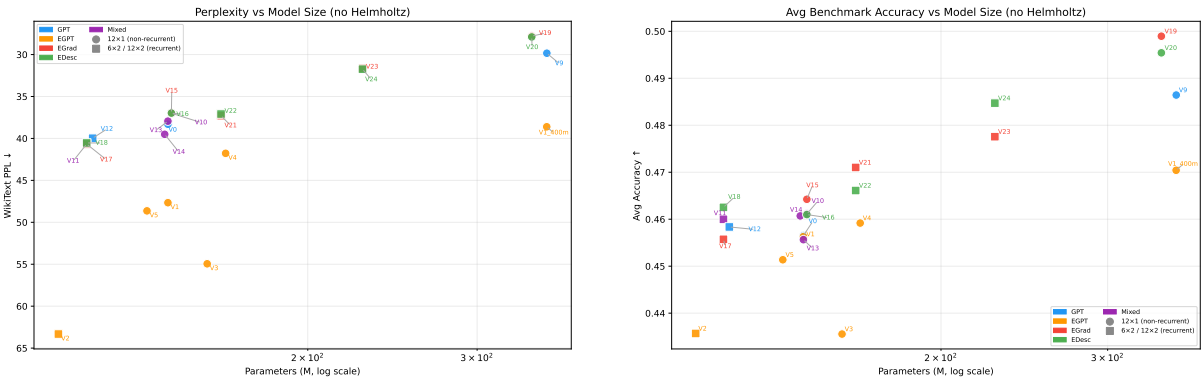


Figure 34: Accuracy vs. parameters — all runs including Mixed (grey). Same caveat as above: grey points are mislabeled runs included for comparison only.



(a) PPL vs. params (log x, no Helmholtz), early runs V0–V18.

(b) Avg accuracy vs. params (log x), early runs V0–V18.

Figure 35: Parameter-efficiency across early variants (V0–V18). Recurrent models (squares) cluster at smaller unique parameter counts.

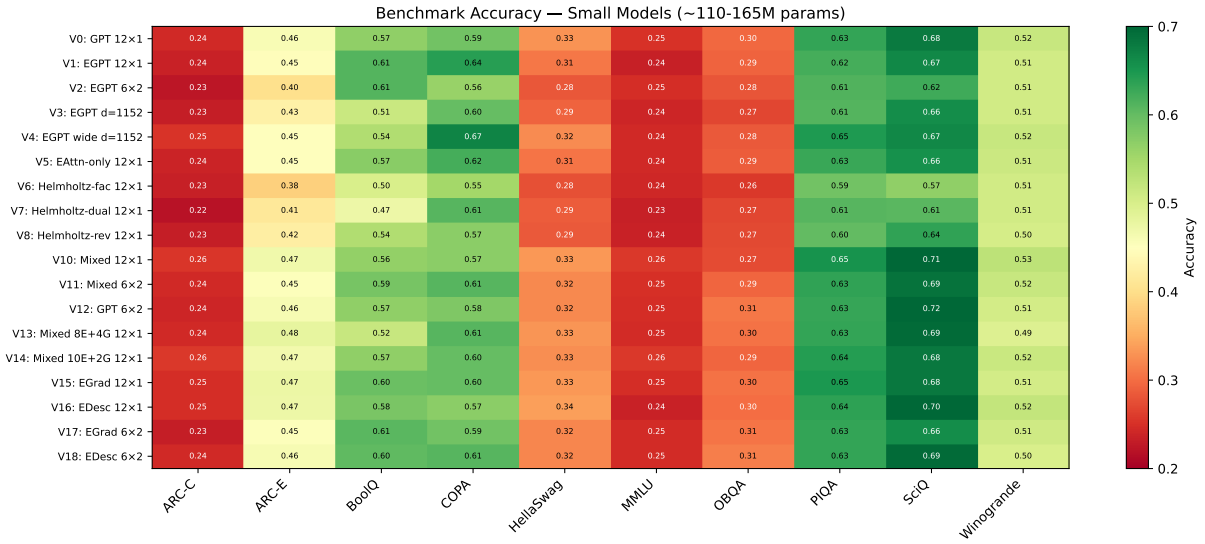


Figure 36: Benchmark accuracy heatmap for all small-scale variants (V0–V18). V15 EGrad and V16 EDesc occupy the top rows alongside V0 GPT.

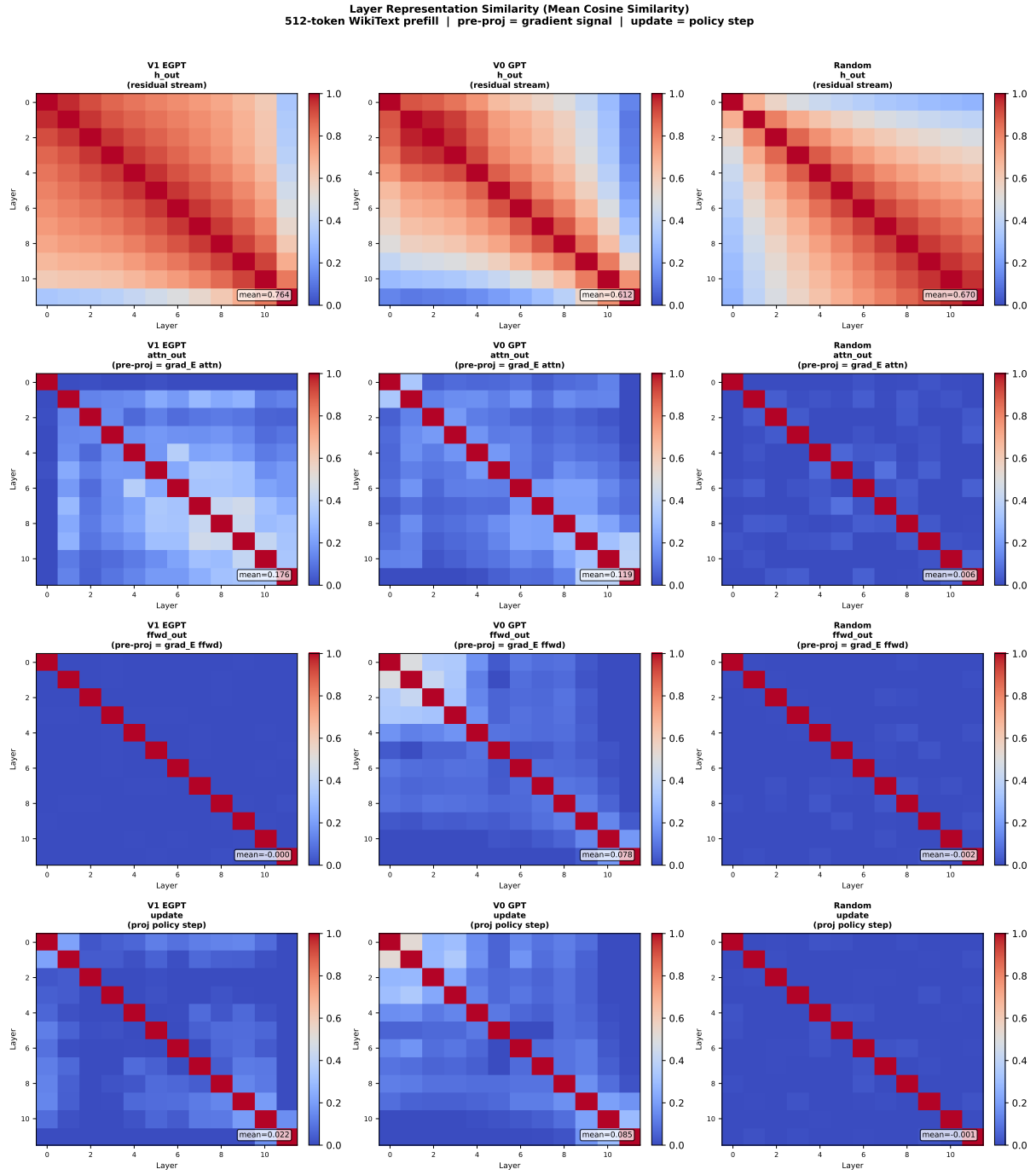


Figure 37: Full 12×12 inter-layer cosine similarity matrices. V1 EGPT attn_out panel shows the highest off-diagonal values.

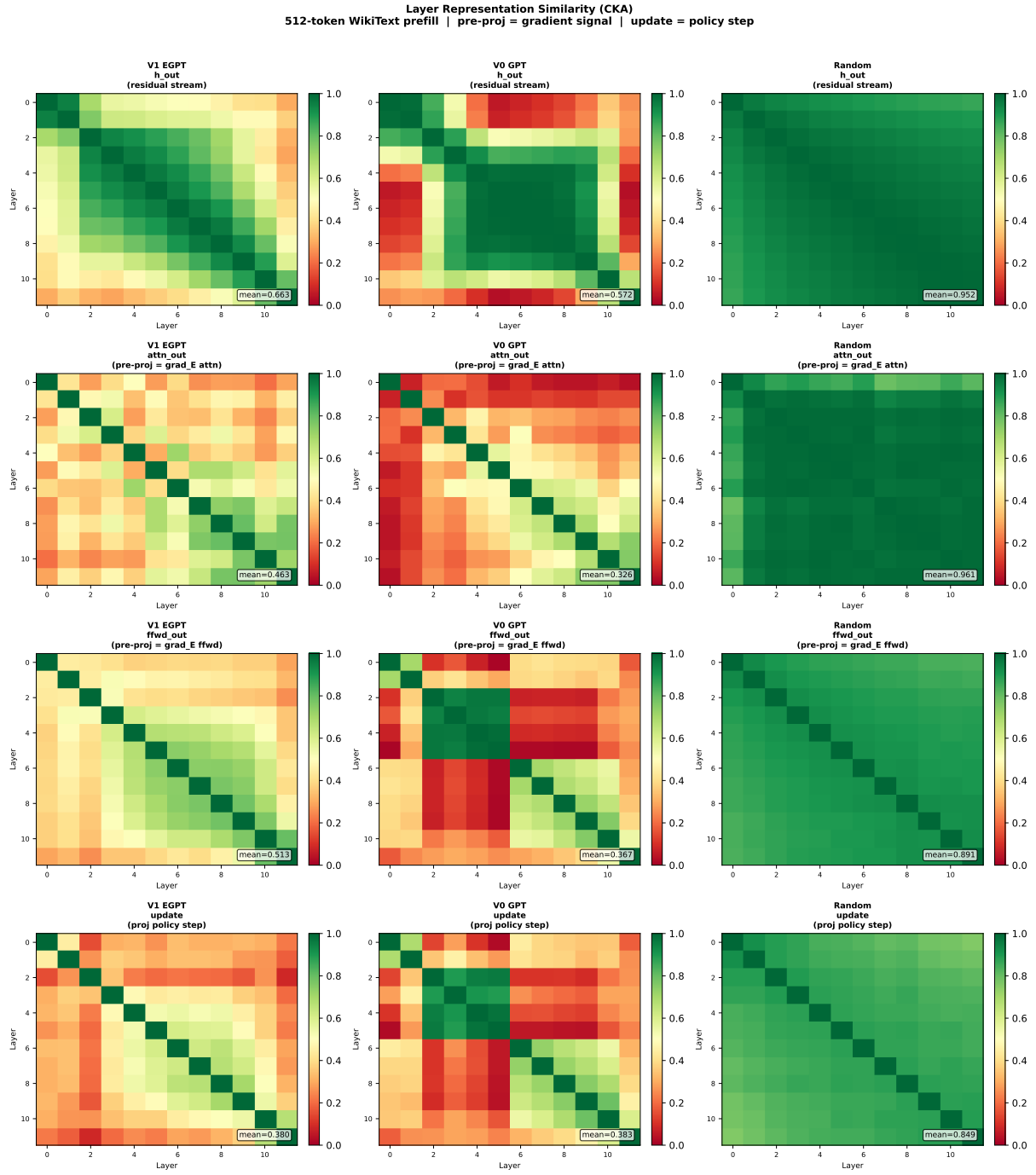
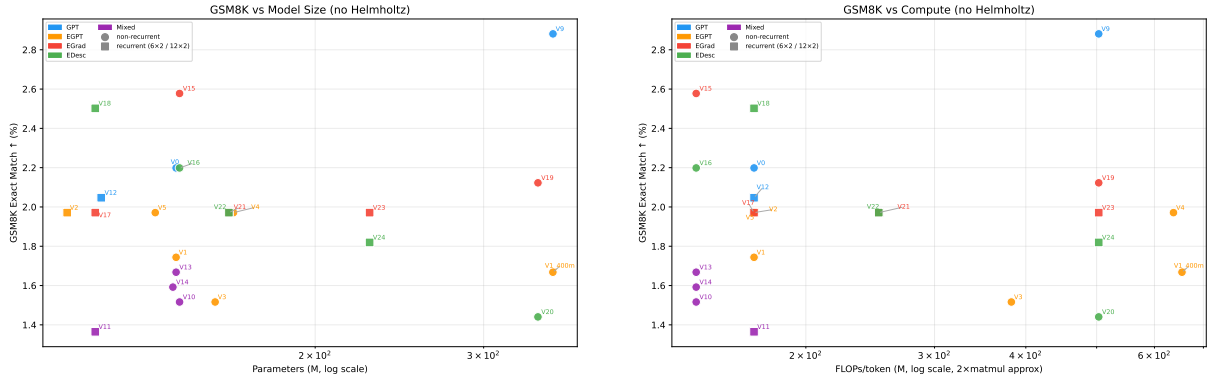


Figure 38: Full 12×12 Linear CKA matrices. The random model has the highest CKA, confirming CKA is dominated by the shared residual stream and is a poor probe.



(a) GSM8K vs. parameters.

(b) GSM8K vs. FLOPs/token.

Figure 39: GSM8K exact-match accuracy across all models. All scores <3%; no architecture consistently leads. V9 GPT (2.9%) and V15 EGrad (2.6%) lead at 354M and 144M scale respectively.